

Data Augmentation for Steel Classification

Bachelor Thesis

Jonathan Püschel

Matrikelnr.: 3539037
E-mail: jonathan.pueschel@web.de
Major: Informatics B. Sc.

Prof. Dr. Thorsten Papenbrock

June 29, 2024

Abstract

Available and affordable construction materials, such as steels, have become scarce. Along with the imminent climate change, this leads to the need to rethink renewal measures in construction, to create a sustainable and economic alternative. One approach to achieve this is to place more emphasis on the preservation and recondition of materials in long existing structures and buildings. Depending on the material type, different forms of maintenance and recondition can be applied.

Steel is a fundamental component for structures in numerous engineering branches. Often steel is essential to ensure stability and durability in constructions, such as bridges and buildings.

Maintenance and recondition, to ensure safety, would be a valid alternative to renewing existing steel entirely. Since different steel types have class-specific characteristics, they require different maintenance procedures, which is why the correct classification of the steel type is crucial.

This is only one of many use cases for steel classification. Various industries, like manufacturing and automotive, could strongly benefit. It could assist with a large number of problems, such as quality control or material selection.

This thesis explores the application of Data Augmentation techniques, aiming to improve the classification abilities of supervised learning models on imbalanced steel datasets commonly found in the steel industry. We investigate the performance of Generative Adversarial Networks (GANs), Wasserstein Generative Adversarial Networks (WGANs) and Synthetic Minority Over-sampling Technique combined with Edited Nearest Neighbors (SMOTE-ENN).

While the traditional GAN failed to capture the full variety of steel classes, the superior WGAN managed to augment realistic data samples. SMOTE-ENN effectively balanced the data by adding samples to minority classes and removing noisy data. The performance of supervised learning algorithms, evaluated with the overall accuracy and f1-scores for each steel class can be significantly enhanced significantly with the application of the Data Augmentation techniques.

Summary

This thesis investigates Generative Adversarial Networks (GANs), Wasserstein Generative Adversarial Networks (WGANs) and Synthetic Minority Over-sampling Technique with Edited Nearest Neighbors (SMOTE-ENN) as Data Augmentation techniques for balancing datasets.

The research demonstrates that traditional GANs experience mode collapse in the context of steel classification, leading to low-quality and unrealistic data. WGANs address this issue effectively by recreating the underlying data distribution, thereby creating new satisfactory data samples. The application of a gradient penalty in the training process of the WGAN is found to be unnecessary and time-consuming for steel data.

SMOTE-ENN proves effective for resampling data by over-sampling minority classes and removing noisy and ambiguous data samples.

The purpose of Data Augmentation is to solve two main problem cases, first, to balance the overall dataset. On an exemplary dataset, the use of the Data Augmentation techniques led to an improvement of the overall accuracy of 10% to 12% with a consistent rise in f1-scores for the majority of steel classes.

The second purpose was to enhance singular steel classes, where supervised learning has difficulties classifying. While the Data Augmentation techniques can enhance the steel class and increase the f1-score, the performance on previously high-performing classes potentially decreases. Accordingly, the application for such cases has to be evaluated extensively.

SMOTE-ENN and WGANs exhibit technique specific limitations. Whilst WGANs create high-quality data, including in high dimensions, they require substantial training time, which can be impractical for real-time application. Despite the fact that SMOTE-ENN is time efficient, the technique performs poorly on high dimensional data, due to the loss of meaningful distances. The choice of the Data Augmentation technique can be based on the specific requirements for the classification task of the system's user. Potential limitations of one Data Augmentation technique are compensated for by the other.

Overall, integrating WGANs and SMOTE-ENN into the steel classification software can significantly improve the performance of supervised learning on heavily imbalanced datasets.

Contents

1 Introduction to Steel Classification **1**
- 1.1 Provided data . . . 1
- 1.2 Evaluation of provided supervised learning algorithms . . . 2
  - 1.2.1 Extreme Gradient Boost . . . 2
  - 1.2.2 Sequential Neural Network . . . 5
- 1.3 Conclusion of Evaluation . . . 8

2 Related Work **9**
- 2.1 Definition of Data Augmentation . . . 9
- 2.2 Generative Adversarial Networks (GANs) . . . 9
  - 2.2.1 Introduction to Generative Adversarial Networks . . . 9
  - 2.2.2 Theory behind GANs . . . 9
- 2.3 Wasserstein GANs . . . 12
  - 2.3.1 Introduction and Theory of Wasserstein GANs . . . 12
  - 2.3.2 Gradient Penalty . . . 14
- 2.4 Synthetic Minority Over-sampling Technique (SMOTE) . . . 16
  - 2.4.1 Introduction and Theory of SMOTE . . . 16
  - 2.4.2 SMOTE and ENN . . . 17
- 2.5 Conclusion . . . 18

3 Application of Data Augmentation on Singular Classes **19**
- 3.1 Generative Adversarial Network . . . 21
  - 3.1.1 Implementation . . . 21
  - 3.1.2 Evaluation of Augmented Data . . . 23
- 3.2 Wasserstein GAN . . . 26
  - 3.2.1 Implementation . . . 26
  - 3.2.2 Evaluation of Augmented Data . . . 28
  - 3.2.3 Implementation of Gradient Penalty . . . 31
  - 3.2.4 Augmentation of St 48-TH with WGAN . . . 33
- 3.3 SMOTE-ENN . . . 37
  - 3.3.1 Implementation and Evaluation of Augmented Data . . . 37
- 3.4 Influence of augmented Data on supervised learning . . . 39
  - 3.4.1 Augmented Data with WGAN . . . 39
  - 3.4.2 Augmented Data with SMOTE-ENN . . . 43
- 3.5 Conclusion . . . 46

4 Application of Data Augmentation on Overall Imbalanced Data **47**
- 4.1 Exemplary dataset . . . 47
- 4.2 Influence of Data Augmentation on supervised learning . . . 47
- 4.3 Conclusion . . . 54

5 Discussion **55**
- 5.1 Limitations . . . 55
- 5.2 Future Work . . . 56
- 5.3 Conclusion . . . 57

1 Introduction to Steel Classification

1.1 Provided data

This thesis is predicated on an ongoing investigation focused on steel classification, employing supervised and unsupervised machine learning methods. Since only supervised learning methods provide actual predictions within this investigation, the emphasis of this work will be placed on these algorithms. The provided labeled steel samples were extracted from railroad bridges and contain the percentage by mass of each element found in the sample. Nine elements (C, Si, Mn, P, S, Cr, Ni, Al, Cu) determine one of six distinct steel types (St 37, St 48, St 48-TH, St Si (silicon steel), puddling steel). Containing 1429 labeled samples, the dataset showcases an imbalanced distribution of steel samples grouped by labels, as depicted in Figure 1. With 827 samples, the vast majority of data is labeled St 48, whereas the remaining 602 samples are divided between the other steel types. Especially noteworthy is the relatively low number of silicon steel and St 48-TH samples within the dataset.
The classification of steels is achieved through supervised learning algorithms, trained on the provided dataset.

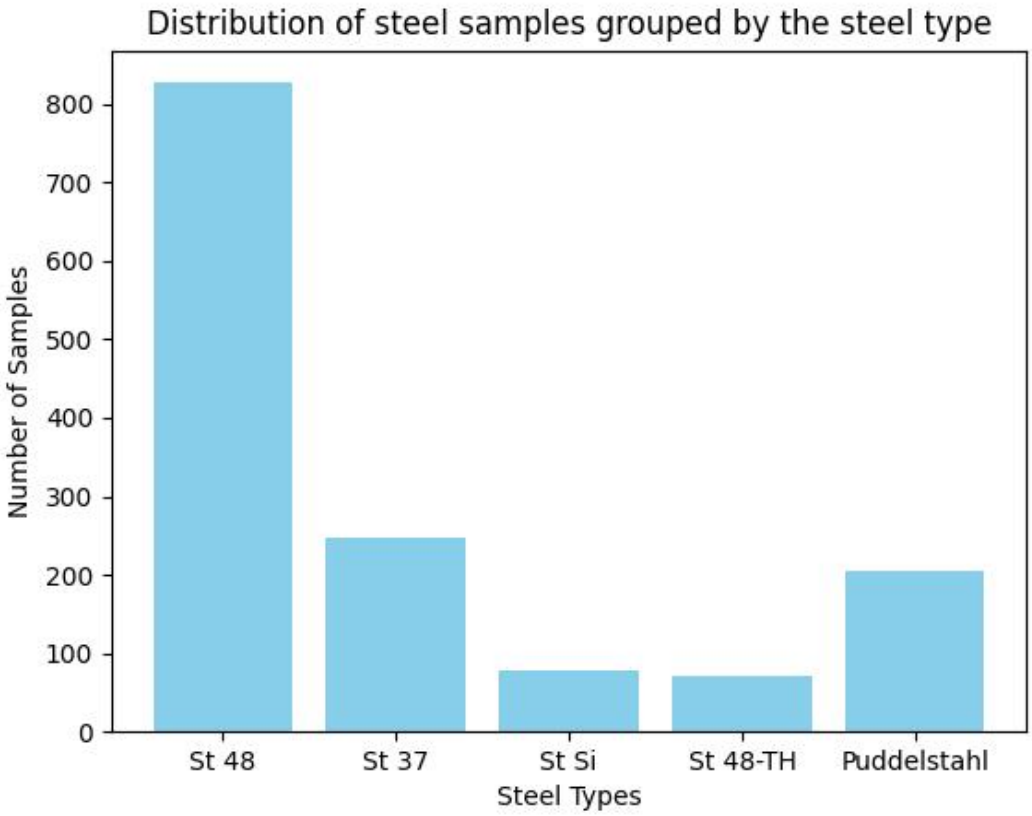


Figure 1: Distribution of steel samples grouped by steel type

1.2 Evaluation of provided supervised learning algorithms

1.2.1 Extreme Gradient Boost

The first algorithm, Extreme Gradient Boost (XGBoost), is an ensemble learning algorithm that utilizes small decision trees (stumps) and implements boosting to achieve high-quality results. Known for its robustness, accuracy and efficiency, XGBoost is especially effective for this classification problem. To train the XGBoost model we use 80% of the dataset, the remaining 20% are used for testing. The implemented XGBoost-classifier reaches an average accuracy after cross validation of 92.87%, which can be enhanced to an accuracy of 95.17%, thanks to hyperparameter optimization.
To acquire a better understanding of how the accuracy is established, we can look into further evaluation measures.
Figure 2 illustrates the confusion matrix of the predictions generated by the XGBoost algorithm. Generally, the predictions exhibit high accuracy with the exception of certain samples. Especially noteworthy is the frequent misclassification of the steels St 37 and St 48-TH, compared to the other steel classes. The confusion matrix indicates that the model faces difficulties clearly differentiating those steel classes. Silicon steel was not misclassified overall, even though the testing set only contained 15 samples of that type. This peculiarity suggests substantial differences in the data between silicon steel and the other steel types.

These observations are further supported by the classification report of the XGBoost model (Figure 3). The evaluation metrics precision, recall and f1-score for St 37 and St 48-TH exhibit the lowest values, with 0.89 and 0.73, compared to the other steel types. This implies that data provided may not be adequate to train a model that classifies those steel types excellently.
For a deeper insight into the generalization capabilities of XGBoost, we approach a dimension reduction involving Principal Component Analysis (PCA). Subsequently, we compare misclassified predictions visually with the ground truth in a coordinate system, as delineated in Figures 4 (a) and (b). Including both training and testing data, it becomes apparent that all steel types are clustered closely. For the majority of points it seems like XGBoost has difficulties predicting the correct steel types at intersections between clusters.

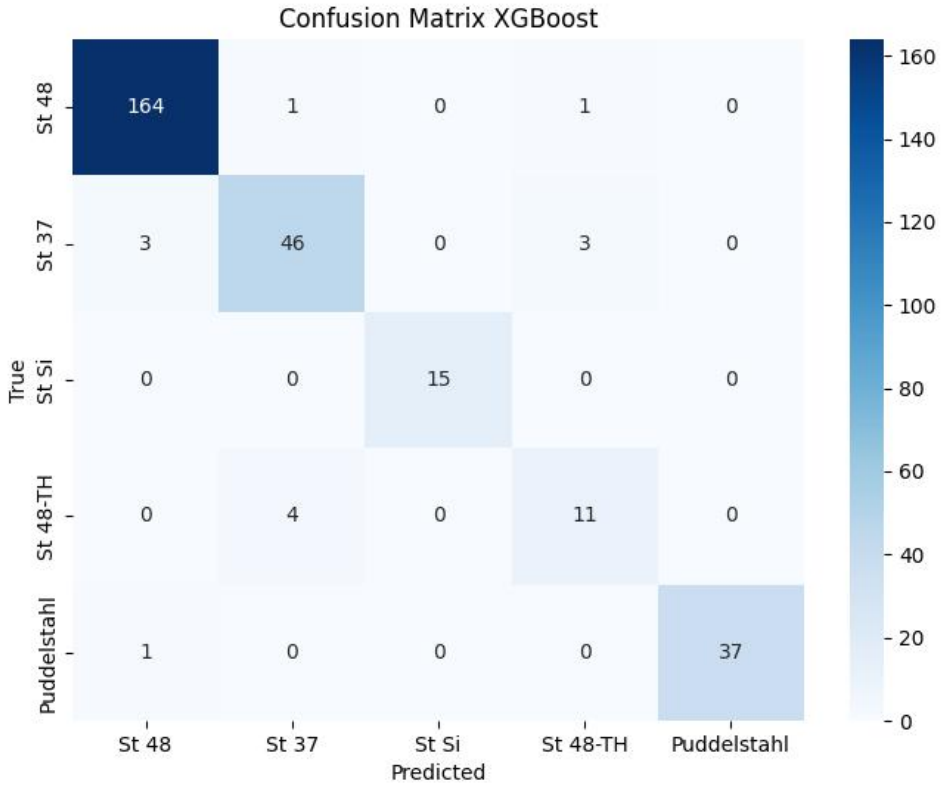
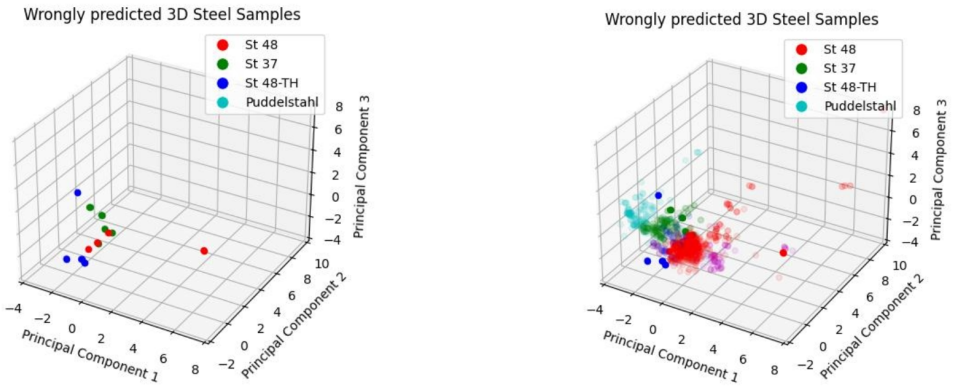


Figure 2: Confusion matrix XGBoost predictions

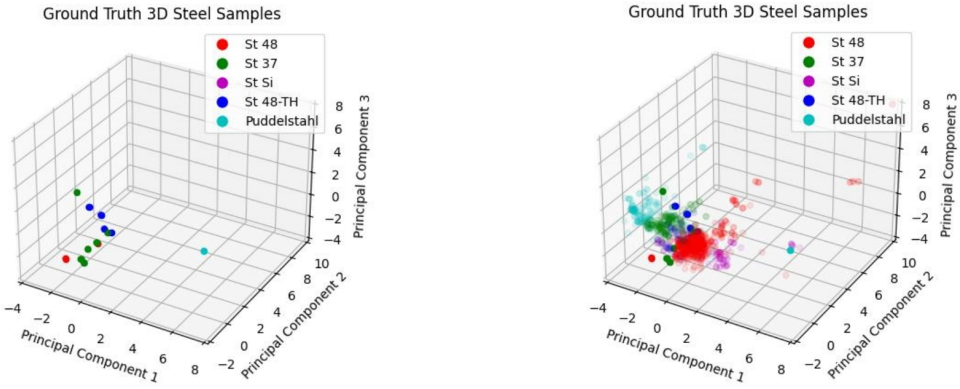
	precision	recall	f1-score	support
St 48	0.98	0.99	0.98	166
St 37	0.90	0.88	0.89	52
St Si	1.00	1.00	1.00	15
St 48-TH	0.73	0.73	0.73	15
Puddelstahl	1.00	0.97	0.99	38
accuracy			0.95	286
macro avg	0.92	0.92	0.92	286
weighted avg	0.95	0.95	0.95	286

Figure 3: Classification report of XGBoost model

Figure 4: Misclassified steel samples XGBoost



(a) Misclassified steel samples XGBoost singularly, and with steel data



(b) Ground truth for misclassified steel samples XGBoost singularly, and with steel data

1.2.2 Sequential Neural Network

The second implemented supervised learning algorithm is a sequential neural network comprising four hidden layers. ReLU activation is used throughout the layers that comprise 248, 124, 64 and 32 neurons. To prevent overfitting, the network implements a dropout layer following the second hidden layer. Additionally, L2 regularization is integrated within the third layer with a regularization value of 0.05. Both regularization measures help to improve generalization and prevent the model from learning noise included in the training data. As an input, the algorithm takes the percentage by mass of each element in the dataset. Predictions of the steel type are established by using the softmax function, a standard function for neural nets for classification problems. To optimize the weights of the edges during the training process, the Adam optimizer is used. As a loss function, we use the Sparse Categorical Cross Entropy to assess the predictive performance of the model during training. Similarly to the XGBoost implementation, the model's performance is estimated using accuracy.

For the training process, 80% of the data provided is used, while the remaining 20% is used for testing. With cross validation, the accuracy of the neural network can be estimated with a value of 95.56%.
The prediction of the testing data is represented in the form of a confusion matrix in Figure 5. Similar to the observations made in the evaluation of XGBoost, the neural net seems to misclassify St 37 and St 48-TH most frequently, while the other steel types are classified almost flawlessly. Especially St 48-TH with a f1-score of 0.75 (Figure 6) shows room for improvement. Investigating the confusion matrix further, the model seems to be unable to make a clear difference between the steels St 37 and St 48-TH for some samples (Figure 5).
Unlike XGBoost, the model seems to face difficulties differentiating certain St 48 samples from St 37. The neural network misclassified St 37 as St 48 eight times, while for XGBoost this type of error only happened once.

For further investigation of misclassified samples, we use dimensionally reduced data and plot them in a three-dimensional coordinate system like previously done for XGBoost (Figure 7). It becomes apparent that like XGBoost, the neural net has difficulties differentiating steel types when samples are in close proximity to each other, such as at intersections of clusters. It is especially noteworthy that the network only incorrectly classifies samples as St 37 and St 48-TH, indicating difficulties in differentiating between those and other steels.

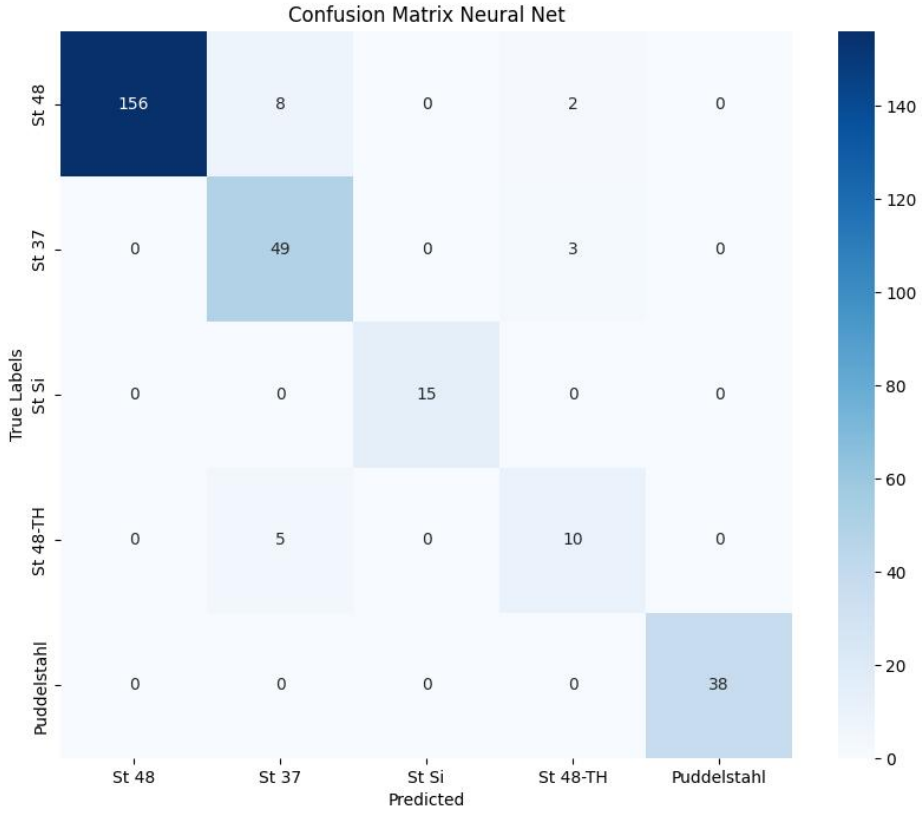
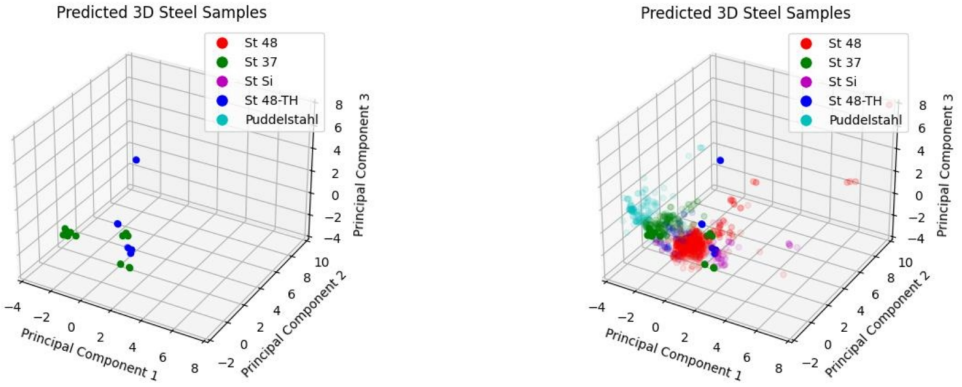


Figure 5: Confusion matrix Neural Network predictions

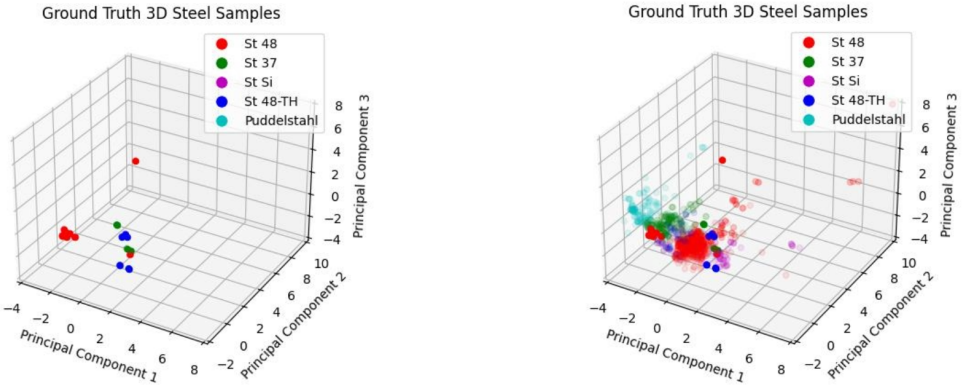
	precision	recall	f1-score	support
St 48	0.98	0.99	0.98	166
St 37	0.94	0.88	0.91	52
St Si	1.00	1.00	1.00	15
St 48-TH	0.71	0.80	0.75	15
Puddelstahl	1.00	1.00	1.00	38
accuracy			0.96	286
macro avg	0.93	0.93	0.93	286
weighted avg	0.96	0.96	0.96	286

Figure 6: Neural Network classification report

Figure 7: Misclassified steel samples Neural Network



(a) Misclassified steel samples Neural Network singularly, and with steel data



(b) Ground truth of misclassified steel samples Neural Network singularly, and with steel data

1.3 Conclusion of Evaluation

Even though externally both supervised learning algorithms seem like they are effective in classification with an accuracy over 95%, after deeper evaluation we discovered areas where the classification could be improved even further. The main issue for the models is that the steel classes St 37 and St 48-TH frequently induce misclassifications, for example due to ambiguity at cluster intersections. One potential explanation for mislabeling these data points could stem from the imbalance in the dataset provided (Figure 1).

By examining the data visually in a coordinate system after reducing the dimensions of data, it is observable that St 37, St 48-TH and St 48 have adjacent clusters. Since there are 827 samples of St 48, 247 samples for St 37 and only 71 samples for St 48-TH, this incongruity might explain the poor performance on minority classes. The exceptions are silicon steel and puddling steel, which seem to have significant differences in their element data, thus making their classification simpler. The imbalance in data can make it challenging for the models to learn key patterns for differentiating minority classes. This can result in a bias towards majority classes and worse performance in generalization.

To balance the data and improve classifications for existing and future data, integrating Data Augmentation techniques into steel classification is a promising strategy. Data Augmentation can artificially expand the size of the dataset. Thanks to the integration of new data points, Data Augmentation can help to fix the imbalance between majority and minority groups in the dataset provided for steel classification. This allows supervised learning models to make more accurate predictions across all steel types.

To integrate useful Data Augmentation techniques into the steel classification software, we consider two main problem cases, derived by the data provided. The first problem is generally imbalanced data, which can lead to poor performance on the classification abilities of supervised learning. Despite the fact that the models provided have satisfactory capabilities after training on imbalanced data, in other cases, for example with less data, the performance might be influenced negatively. To fix this issue, we need Data Augmentation techniques that are capable of balancing the overall dataset.
The second problem is a poor performance of supervised learning on single steel classes, as encountered in the evaluation. Data Augmentation for singular steel classes might be able to enhance the abilities towards that class.

2 Related Work

2.1 Definition of Data Augmentation

Data Augmentation is the process of artificially expanding datasets by applying transformations to the original data. Patterns and semantics in the original data will be preserved in newly augmented data. It covers a spectrum of techniques, including supervised and unsupervised methods. The enhancement of data quantity and fixing imbalances in datasets are only some of numerous applications of Data Augmentation techniques. A sufficient amount of data is essential for the training process of machine learning models. The quality and quantity of data are the most determining factors for the performance of a trained model. Data Augmentation has already been integrated successfully in various fields, like image generation[MG18], natural language processing[LHC22] and speech recognition[PCZ+19], thus strongly enhancing machine learning abilities.
In the following, we will introduce Data Augmentation techniques that are applicable to the steel data provided.

2.2 Generative Adversarial Networks (GANs)

2.2.1 Introduction to Generative Adversarial Networks

Generative Adversarial Networks (GANs) were introduced in 2014 and implement two deep learning networks, the generator and the discriminator, working against each other[GPAM+20]. The discriminative model takes data as an input and determines if the samples fit into the original dataset. The generative model creates fake samples derived by given data. To make improvements, both models compete against each other. The generator's output will be received and judged by the discriminator. If the discriminator uncovers the fake sample, the capabilities of the generator model will be improved. If the discriminator misjudges the sample as real, the discriminator model will be improved. Due to the adversary between networks, they can force advancements, resulting in realistic and plausible data created by the generator.

2.2.2 Theory behind GANs

In the original paper, the generator G and the discriminator D are trained on the following loss function or value function:

$$\min_G \max_D V(G,D) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

$\mathbb{E}_{x\sim p_{\text{data}}(x)}$ is the probability that the discriminator recognizes real data, $\mathbb{E}_{z\sim p_z(z)}$ is the probability of the discriminator recognizing fake data. $x \sim p_{\text{data}}(x)$ is the data distribution of real data samples. $z \sim p_z(z)$ represents random data taken from a noise distribution, which will be the input for the generator. The generator will forge fake

samples out of the given noise. For best possible performance, the discriminator model aims to maximize the value function, while the generator seeks to minimize it. Since the generator is only involved in the second part of the equation, the first part will be canceled out during minimization because it is handled as a constant. For optimization of the value function, the paper suggests using stochastic gradient descent, resulting in the following algorithm for the training of a Generative Adversarial Network.

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Figure 8: Training algorithm for GAN[GPAM+20]

The algorithm depicted in Figure 8 follows a two-step process. Firstly, the discriminator is trained by increasing the stochastic gradient with real data and fake data supplied by the generator, built out of noise used as the input. Secondly, the generator is trained using stochastic gradient descent by generating fake data samples from noise and passing them through the generator. The resulting fake samples are fed to the discriminator. The generator's weights are updated to create improved fake samples, which are more likely to fool the discriminator. This iterative process continues updating generator and discriminator competitively for a given number of training iterations. With optimization, the generator is supposed to find the global minimum of the value function. In that case, the generated distribution of data p_{g} is equal to the distribution of the original data p_{data}, as proven in the original paper[GPAM+20].

After the training process, the generator is able to create realistic samples derived by the given dataset. By outputting a value between 0 and 1, the discriminator will be able to classify, if a given sample is real data, or not. Values close to 1 indicate a real sample, while values close to 0 indicate a fake sample. GANs are a powerful tool for Data Augmentation, since a well-trained generator can create synthetic data that closely resembles real data.

Successful related work with GANs on tabular data

Even though GANs are especially popular for image generation, they have been successfully applied to the generation of tabular data. One notable work is the development of Tabular GAN (TGAN), based on Long Short-Term Memory networks without convolutional layers[XV18]. The TGAN has been tested on multiple datasets, showing realistically generated data that represents the underlying distribution of original data. On the Census dataset, a TGAN managed to preserve the correlation better than other traditional statistical methods.
Despite the fact that GANs have been successfully applied to tabular data, the training of GANs presents several difficulties. The generator might produce a limited variety of outputs, not representing the full dataset, resulting in a bias within augmented data[ACB17]. Another problem can occur if the discriminator becomes too effective, causing vanishing gradients in the generator. In this case, the generator is not able to improve its output, which will lead to stagnation in the learning process.
In a GAN, it can be difficult to derive a meaningful gradient, since the generator is performing poorly in the beginning of training. As a divergence measure, GANs use Kullback–Leibler divergence and Jensen–Shannon divergence[ACB17]. Both measures neglect the distance between two probability distributions. In case of GANs, the distance between the generated distribution p_{g} and the distribution within real data p_{data} also provides information about how well the generator is performing, which is why it should not be neglected.

2.3 Wasserstein GANs

Introduced in 2017, Wasserstein GANs extend the concept of standard GANs and provide solutions for previously explained problems. In the following, we replace the term "discriminator" with "critic". Modifications in the functionality of the discriminator motivated introducing the new terminology for the network[ACB17].

2.3.1 Introduction and Theory of Wasserstein GANs

The architecture of a Wasserstein GAN (WGAN) is analogue to a GAN. Unlike for standard GANs, WGANs use the Wasserstein Distance, also known as Earth-Mover's Distance, to determine the distance of distributions $W(p_\mathrm{r}, p_\mathrm{g})$, with p_r as the distribution of real data. It is defined as the minimum cost of transporting mass to convert one distribution into the other. The cost is estimated by the distance moved times the amount of mass. Wasserstein Distance can be expressed as

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(x,y)\sim\gamma}[||x - y||]$$

where $\Pi(p_r, p_g)$ represents the set of all joint distributions γ whose marginals are p_r and p_g[ACB17]. The resulting value is the minimum cost required to transform one distribution into the other. The expectation $\mathbb{E}$ is taken over the cost of transporting mass from x to y.
Instead of returning a value between 0 and 1 for classification, the critic uses the Wasserstein Distance to formulate a loss function that outputs a real number, representing the authenticity of the input data. The result is the following loss function that calculates the difference between the mean scores assigned to real and fake samples.

$$\mathcal{L} = \max_{\|D\|L\leq 1} \mathbb{E}_{x\sim p_r}[D(x)] - \mathbb{E}_{z\sim p_z}[D(G(z))]$$

The discriminator aims to maximize the distance between real and fake data, thereby maximizing the loss function, while the generator seeks to minimize the distance as well as the loss function.
If the generated samples become indistinguishable, the discriminator has to guess which of the samples are true and which ones are fake. This will lead to the loss function converging to zero. The Wasserstein Distance requires the critic to be 1-Lipschitz continuous, ensuring a meaningful distance and meaningful feedback for the generator. 1-Lipschitz continuity is a concept that characterizes functions depending on their mappings between spaces.
A function $f : X \to Y$ between two spaces X and Y is Lipschitz continuous if there exists a real constant $C \geq 0$ whereby for all x_1, x_2 in X, the following inequality holds:

$$|f(x_1), f(x_2)| \leq C \cdot |x_1, x_2|$$

Since 1-Lipschitz continuity is required, we set the constant value C equal to 1. The enforcement of 1-Lipschitz continuity assists the critic to avoid excessively growing gradients by stabilizing the training. Additionally, it improves the convergence compared

to a standard GAN, thereby fixing issues concerning mode collapse and vanishing gradients.
The alterations to the standard GAN result in the following algorithm for training WGANs.

Algorithm 1 WGAN, our proposed algorithm. All experiments in the paper used the default values $\alpha = 0.00005$, $c = 0.01$, $m = 64$, $n_{\text{critic}} = 5$.

Require: : α, the learning rate. c, the clipping parameter. m, the batch size. n_{critic}, the number of iterations of the critic per generator iteration.
Require: : w_0, initial critic parameters. θ_0, initial generator's parameters.

1: **while** θ has not converged **do**
2: **for** $t = 0, ..., n_{\text{critic}}$ **do**
3: Sample $\{x^{(i)}\}_{i=1}^{m} \sim \mathbb{P}_r$ a batch from the real data.
4: Sample $\{z^{(i)}\}_{i=1}^{m} \sim p(z)$ a batch of prior samples.
5: $g_w \leftarrow \nabla_w \left[\frac{1}{m}\sum_{i=1}^{m} f_w(x^{(i)}) - \frac{1}{m}\sum_{i=1}^{m} f_w(g_\theta(z^{(i)}))\right]$
6: $w \leftarrow w + \alpha \cdot \text{RMSProp}(w, g_w)$
7: $w \leftarrow \text{clip}(w, -c, c)$
8: **end for**
9: Sample $\{z^{(i)}\}_{i=1}^{m} \sim p(z)$ a batch of prior samples.
10: $g_\theta \leftarrow -\nabla_\theta \frac{1}{m}\sum_{i=1}^{m} f_w(g_\theta(z^{(i)}))$
11: $\theta \leftarrow \theta - \alpha \cdot \text{RMSProp}(\theta, g_\theta)$
12: **end while**

Figure 9: Training algorithm for WGAN[ACB17]

The pseudocode provided in Figure 9 of the training algorithm of a Wasserstein GAN works similarly to a normal GAN. The main difference is that the algorithm will terminate as soon as the generator's parameters converge, which is when the loss of the critic goes to zero. Furthermore, for every update of the generator there are five updates to the critic. Another difference is that the algorithm suggests using RMSProp to determine the gradient, instead of using stochastic gradient descent. Additionally, in line 7 weight clipping is applied to the critic's weights to ensure 1-Lipschitz continuity to prevent mode collapse.

2.3.2 Gradient Penalty

The introduction paper on Wasserstein GANs already points out weaknesses of the weight-clipping functionality implemented in the example algorithm (Figure 9). Large clipping parameters are potentially time-consuming due to the increased duration of weights reaching their limit. This leads to difficulties in training the critic until optimality. Furthermore, small clipping can induce vanishing gradients in case of a large number of layers or the absence of batch normalization.
This led to the introduction of gradient penalty for Wasserstein GANs[GAA+17]. With gradient penalty, there is a more effective way to ensure 1-Lipschitz continuity compared to weight clipping. The idea behind gradient penalty is to constrain the gradient norm of the critic to 1 at most to ensure 1-Lipschitz continuity. This is achieved with a penalty to the gradient norm (GP).

$$GP = \lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}}[(||\nabla_{\hat{x}} D(\hat{x})||_2 - 1)^2]$$

Newly introduced is $\hat{x}$, which is an interpolation of data samples determined with the formula

$$\hat{x} = \epsilon x + (1 - \epsilon) G(z)$$

with ϵ being a random number in the interval [0, 1], x being a real sample and $G(z)$ being a fake sample.

In the formula for gradient penalty, λ is the regularization parameter, which is multiplied by the norm of the gradient with respect to $\hat{x}$ subtracted by one and then squared. Since the norm of the gradient will be close to 1, 1-Lipschitz continuity will be enforced.

The gradient penalty is included to the WGAN by adding it to the loss function.

$$\mathcal{L} = \max_{\|D\|L \leq 1} \mathbb{E}_{x \sim p_r}[D(x)] - \mathbb{E}_{z \sim p_z}[D(G(z))] + \lambda \mathbb{E}_{\hat{x} \sim p_{\hat{x}}}[(||\nabla_{\hat{x}} D(\hat{x})||_2 - 1)^2]$$

The algorithm included in the paper on gradient penalty for Wasserstein GANs works similarly to the standard WGAN training algorithm (Figure 10). The loss function

Algorithm 1 WGAN with gradient penalty. We use default values of $\lambda = 10$, $n_{\text{critic}} = 5$, $\alpha = 0.0001$, $\beta_1 = 0$, $\beta_2 = 0.9$.

Require: The gradient penalty coefficient λ, the number of critic iterations per generator iteration n_{critic}, the batch size m, Adam hyperparameters α, β_1, β_2.
Require: initial critic parameters w_0, initial generator parameters θ_0.
while θ has not converged **do**
  for $t = 1, ..., n_{\text{critic}}$ **do**
    for $i = 1, ..., m$ **do**
      Sample real data $\boldsymbol{x} \sim \mathbb{P}_r$, latent variable $\boldsymbol{z} \sim p(\boldsymbol{z})$, a random number $\epsilon \sim U[0, 1]$.
      $\tilde{\boldsymbol{x}} \leftarrow G_\theta(\boldsymbol{z})$
      $\hat{\boldsymbol{x}} \leftarrow \epsilon \boldsymbol{x} + (1 - \epsilon)\tilde{\boldsymbol{x}}$
      $L^{(i)} \leftarrow D_w(\tilde{\boldsymbol{x}}) - D_w(\boldsymbol{x}) + \lambda(\|\nabla_{\hat{\boldsymbol{x}}} D_w(\hat{\boldsymbol{x}})\|_2 - 1)^2$
    end for
    $w \leftarrow \text{Adam}(\nabla_w \frac{1}{m} \sum_{i=1}^{m} L^{(i)}, w, \alpha, \beta_1, \beta_2)$
  end for
  Sample a batch of latent variables $\{\boldsymbol{z}^{(i)}\}_{i=1}^{m} \sim p(\boldsymbol{z})$.
  $\theta \leftarrow \text{Adam}(\nabla_\theta \frac{1}{m} \sum_{i=1}^{m} -D_w(G_\theta(\boldsymbol{z})), \theta, \alpha, \beta_1, \beta_2)$
end while

Figure 10: Training algorithm for WGAN with GP[GAA+17]

is updated, now including the penalty for the gradient with a regularization value, lambda. To optimize the loss, the algorithm suggests using the Adam optimizer, without momentum ($\beta_1 = 0$), instead of RMSProp.

Successful related work with WGANs on tabular data

Tabular data has been successfully generated with technologies based on WGANs. One notable work based on WGANs is the application of a conditional WGAN to credit scoring datasets, containing numerical and categorical data[EL21]. This study developed a conditional WGAN that demonstrated superior performance on several credit scoring datasets compared to oversampling methods, such as Synthetic Minority oversampling.

2.4 Synthetic Minority Over-sampling Technique (SMOTE)

2.4.1 Introduction and Theory of SMOTE

Synthetic Minority Over-sampling Technique (SMOTE) is used in machine learning to balance minority classes in imbalanced datasets. Over-sampling is a method for machine learning that fixes imbalances by creating new synthetic data. The algorithm picks samples of the minority class and creates new synthetic samples along the line segments in the direction of the k nearest neighbors[CBHK02].
Generated samples are calculated by taking the difference of the feature vector and a nearest neighbor, multiplying it with a random number between 0 and 1 and then adding it back to the feature vector.

$$V_{new} = (V_F - V_{NN})\ rand(0,1) + V_F$$

These steps can be broken down into the following algorithm:

Algorithm 1 SMOTE (N, V_F, k)
Input: N is the number of new samples to augment, V_F is the feature vector, k is the number of neighbors
Output: a set N_{new} with N newly augmented samples

Initialize an empty set N_{new}
for $n = 1$ to N **do**
    Randomly select a sample V_F from the minority class
    Find the k nearest neighbors of V_F in the feature space
    V_{NN} = Randomly select one of the k nearest neighbors
    $V_{new} = (V_F - V_{NN})\ rand(0,1) + V_F$
    Append V_{new} to N_{new}
end for
return N_{new}

In the case of multiple minority classes, like in the steel data, the algorithm is executed for each minority class.
Thanks to over-sampling with SMOTE, minority classes can be balanced, resulting in better generalization and reduced bias towards majority classes from supervised learning algorithms.
By generating new data, new problems can occur. It can lead to overfitting to the data, overlapping samples and noise samples[XSNK20]. As a fix, SMOTE can be used with under-sampling methods. Under-sampling is a method in machine learning that fixes imbalances by removing existing data to make it more generalizable.

2.4.2 SMOTE and ENN

Edited Nearest Neighbor (ENN) is a technique that implements under-sampling to fix class imbalance problems by removing instances of the majority class that is classified differently by the k-nearest neighbor algorithm[Wil72]. Removing noisy and misclassified instances leads to enhanced quality of the dataset and improved performance of machine learning algorithms.
ENN determines the k-nearest neighbors for each feature vector in each majority class. If more neighbors are from a different class than V_F, V_F will be removed.

Algorithm 2 ENN (mjc, k)
Input: mjc is a majority class, k is the number of neighbors

```
for V_F in mjc do
    Find the k nearest neighbors of V_F in the feature space
    if neighbors from other class are predominant then
        remove V_F from ds
    end if
end for
return N_new
```

This algorithm is repeated for every majority class of the given dataset. The for-loop can be altered depending on the new proportion needed.

The SMOTE-ENN method combines both algorithms for maximum efficiency[BPM04]. In the combination of SMOTE and ENN, ENN is adjusted so that it is not only applied to majority class, but to all classes in the dataset.
First, SMOTE generates new synthetic samples for minority classes, then ENN removes all samples that have more neighbors whose classes differ from the sample's class. With the combination of both methods, problems like overlapping samples and noise samples can be eliminated, resulting in cleaner synthetic data.

Successful related work with SMOTE-ENN on tabular data

Studies have demonstrated the practicality of using SMOTE-ENN for handling imbalances of tabular data. One application is in the field of medical diagnosis. A hybrid algorithm of misclassification-oriented SMOTE and ENN has been particularly successful[XSNK20]. This approach managed to balance the dataset and improved the classification accuracy significantly on the dataset.

2.5 Conclusion

GANs have been successfully implemented for generative modeling in various areas. They make it possible to create realistic data samples through adversarial training between two neural networks. While traditional GANs have shown to be effective, instability in training and mode collapse often limit the full potential.
Wasserstein GANs provide a new approach to solve those issues by introducing the Wasserstein distance to the system for more stability during the training process. To further enhance the capabilities of WGANs, the concept of gradient penalty can be integrated to enforce 1-Lipschitz continuity for the critic.
As a Data Augmentation technique, Wasserstein GANs with gradient penalty provide a robust, high-quality approach with promising abilities for Data Augmentation that can be applied to steel classification.
SMOTE-ENN addresses imbalances by generating samples with the creation of interpolations between a feature vector and its k nearest neighbors and removes noisy and ambiguous data.
All data augmentation techniques presented in this section offer promising abilities to enhance the classification abilities of supervised learning models.

3 Application of Data Augmentation on Singular Classes

To address the issue discussed in Section 1 of misclassifying St 37 and St 48-TH, we plan to augment new data for both minority classes. Since the augmentation techniques will function independently of the steel class, in this section we will exemplary evaluate augmented St 37-samples in broad detail, created by the presented techniques. The augmentation of the other minority class St 48-TH will be summarized briefly due to identical observations.
During the evaluation of the predictions of the classification models in Section 1, we established that both algorithms seem to have difficulties making valid predictions for both steels.
As metrics to assess the quality of augmented samples, we plot new samples in a 3D coordinate system with PCA and compare the distributions of each element amount between hte original and augmented data. Additionally, we compare the true underlying distribution of values for each element (Figure 11) to the distribution of augmented data samples. To gain an idea about the variance difference of true and fake data, we also examine the ranges of values in both datasets.
Within the next subsections, we will be referring to the following ranges of the original dataset:

Range of C: (0.0164, 0.269666666666667)

Range of Si: (0.001, 0.29)

Range of Mn: (0.28, 0.682)

Range of P: (0.0035, 0.0731)

Range of S: (0.0111666666666667, 0.1)

Range of Cr: (0.005, 0.109)

Range of Ni: (0.01485, 0.107166666666667)

Range of Al: (0.0007, 0.0510666666666667)

Range of Cu: (0.0069333333333333, 0.285111111111111)

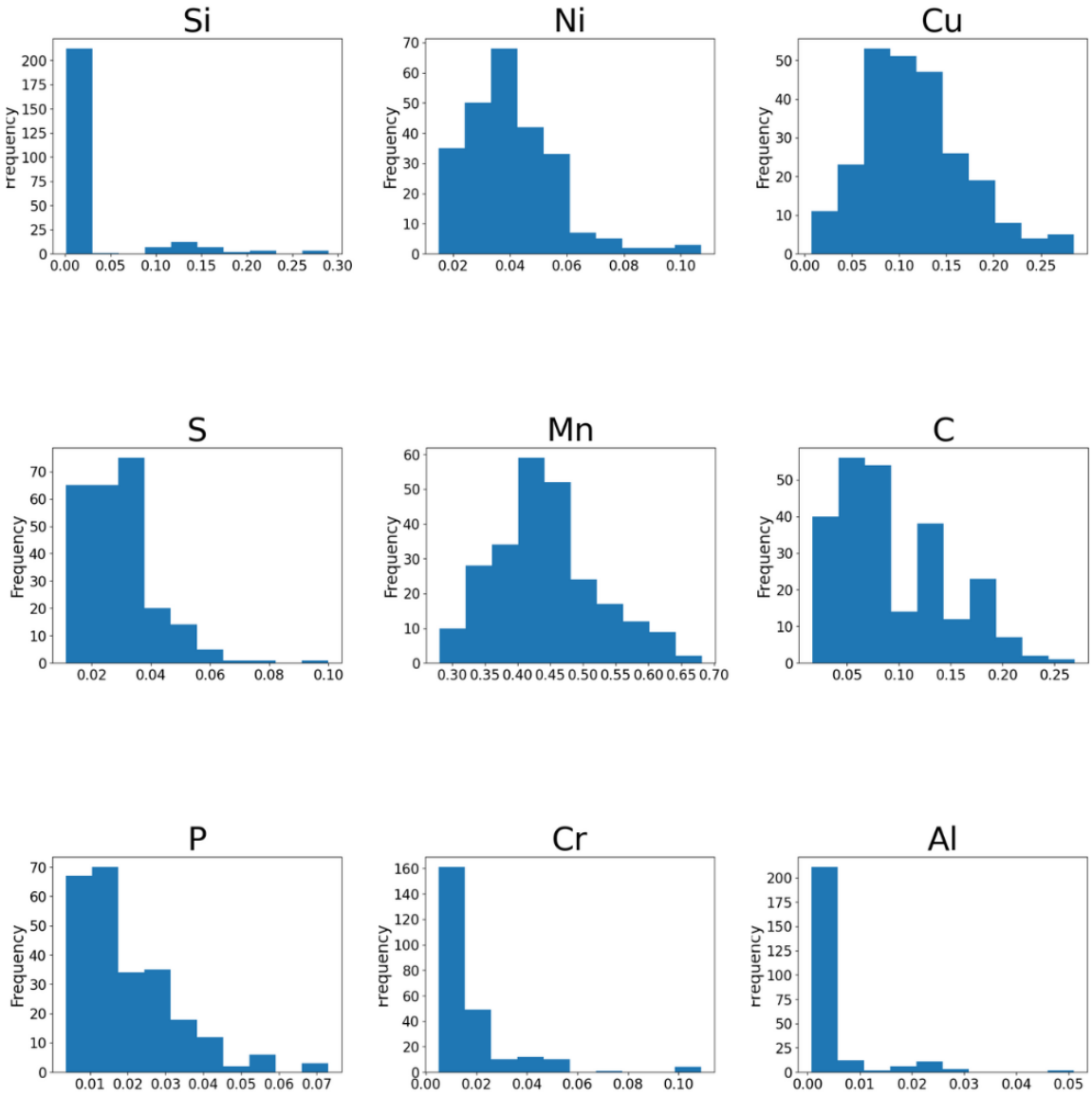


Figure 11: Distribution of each element amount of St 37 in the real dataset

3.1 Generative Adversarial Network

3.1.1 Implementation

Implementing a GAN for steel classification brings challenges. GANs are especially popular for image generation. For human eyes, it can be easily distinguishable if an image created by the generator looks realistic and fits to the provided data. With our steel dataset, it is difficult to estimate whether generated data is plausible or not. The generated weights by element of each sample need further investigation to gain an unambiguous answer. This is why we have to investigate loss functions and outputs carefully to ensure that the GAN truly provides meaningful results.

To ensure that the generator and discriminator have similar complexities, the architecture of both networks will be almost identical. We implement both as sequential models with three hidden layers. Hidden layers one and three comprise 128 neurons with ReLU activation, while the second hidden layer is a dropout layer for regularization. Latent noise, serving as the generators input, is set to a shape of (10, 1). The output of the generator will be a steel sample comprising nine element parameters. The discriminator will output a value between 0 and 1. The closer the value is to 0, the higher is the possibility for the input sample to be fake. To derive the same functionality of the loss function as presented in the original GAN paper, we use binary cross-entropy loss, which we optimize with the Adam optimizer.
For training the GAN, we add each sample, misclassified by supervised learning (Section 1), seven additional times to enhance the representation of similar samples in the augmented dataset. During the training process, we set hyperparameters that can be changed depending on the performance of the model. To keep the process at low cost, we will train the model until convergence for 1000 epochs with a batch size of 64. Figure 12 shows the learning process in the form of loss curves of the implemented GAN.

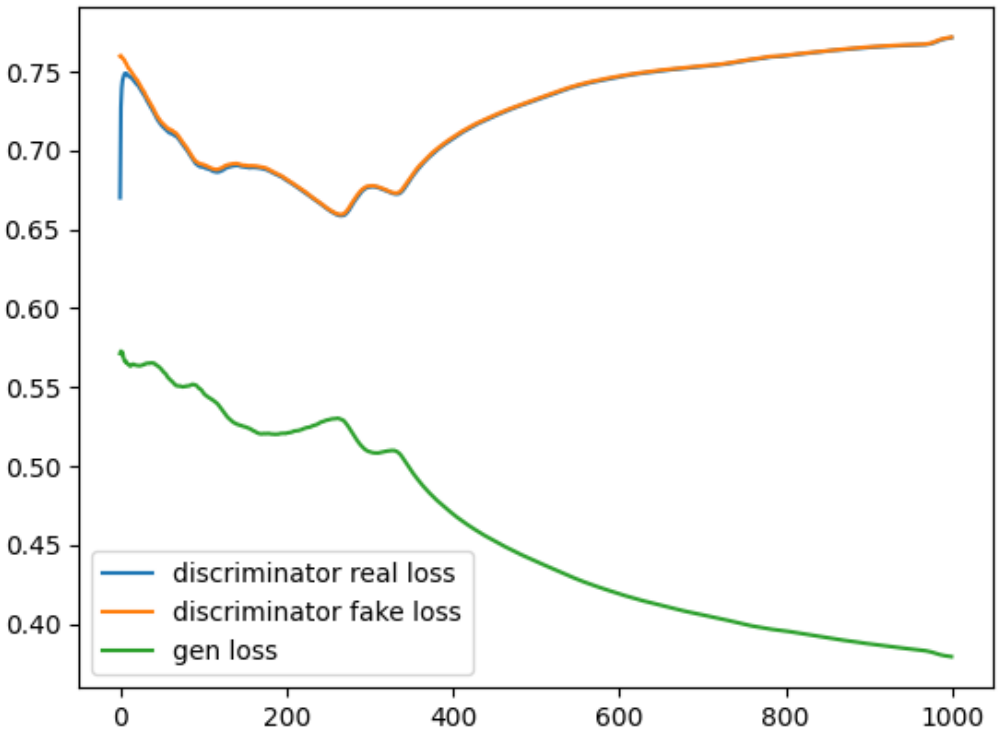


Figure 12: Loss curves of discriminator and generator in a GAN

As depicted in Figure 12, the loss curves of the discriminator and generator show similar behavior, since the curves are mirrored on the x-axis. This suggests that the models made alterations as a reaction to changes of the competitor. Furthermore, the convergence of both graphs indicates that a balance between generator and discriminator has been established during training.
By contrast, the fact that the discriminator's loss curve for real and fake samples is nearly always equal suggests that from the beginning of training it struggles differentiating between the samples, resulting in poor feedback for the generator.

For a well-trained GAN, we would expect the discriminator's loss of the real samples to decline over time, while the loss of the fake samples inclines. This would show that the discriminator has found a way to better distinguish fake samples. For the loss of the generator, the curve would gradually decline, indicating improvements in the generating abilities.
Since the generator's loss steadily declined over time, the augmentation functionality of the generator improves. To gain a degree for the quality of the samples, further investigation of samples is needed.

3.1.2 Evaluation of Augmented Data

To gain a better impression of the quality of the training, we evaluate 100 augmented samples, created by the generator. We scale and dimensionally reduce the augmented data to three dimensions with the help of PCA for visualization in a coordinate system (Figure 13).

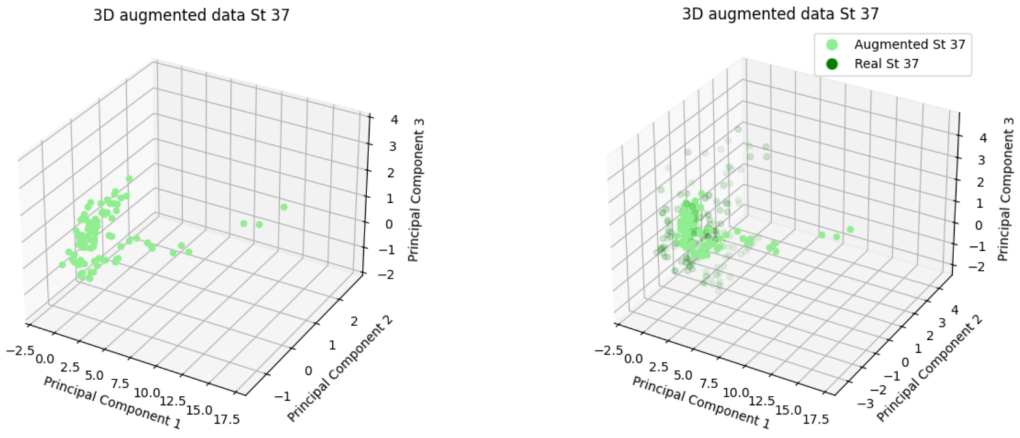


Figure 13: Augmented St 37 samples by GAN

The GAN manages to augment visually plausible new St 37 samples. Most of the newly augmented samples are within the general cluster of St 37 of the real data. It becomes apparent, that many samples have a small distance to each other, thereby not covering the whole variance within the real dataset. A potential explanation could be that the generator overfits to the data, thereby not covering the full variance of the data (mode collapse[LSBL18]). This would also explain the loss curve of the discriminator in Figure 12, since the discriminator does not seem to be able to tell apart real and fake samples from the beginning. Because of that, at some point during training, the generator did not receive any more meaningful feedback from the discriminator, which led to the stagnation in the augmentation variance.
Even though most augmented samples fit into the cluster visually, we can already distinguish outliers that are not suited for the St 37 steel class (Figure 14).

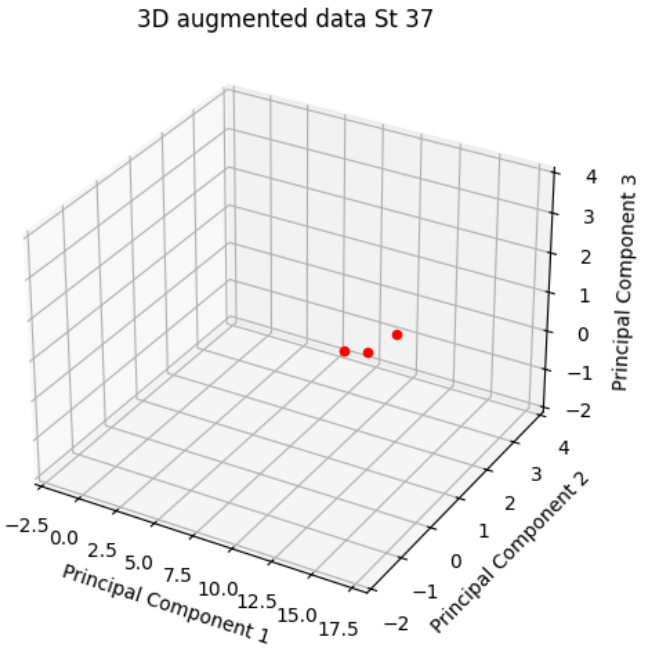


Figure 14: Outliers augmented St 37 samples by GAN

For further evaluation, we pass the augmented data through the supervised learning algorithms provided. Both models classify the newly augmented samples flawlessly, even the outliers, again suggesting mode collapse in the GAN.
Comparing the distribution of each element amount of augmented St 37 data (Figure 15) to original data (Figure 11), we observe poor recreational performance of Ni, C and S. Instead of learning the distribution within the data, the GAN seems to assert low values to those elements. This suggests that low values for the elements are clear indicators for St 37. The ranges of values for Ni, C and S also support this statement when compared with the original ranges.

Range of Ni: (1.0348314e-06, 0.002048513)

Range of C: (0.0005072948, 0.03505194)

Range of S: (9.092107e-08, 0.00057515246)

The GAN manages to generate realistic St 37 samples. The generated data does not depict the full variety of St 37 samples indicating a mode collapse within the model. For augmenting steel data, we want new diverse data that represents the full spread of possible St 37 samples so supervised learning algorithms can better differentiate between steel types. Even though the GAN augments St 37 data, this expectation is not met.

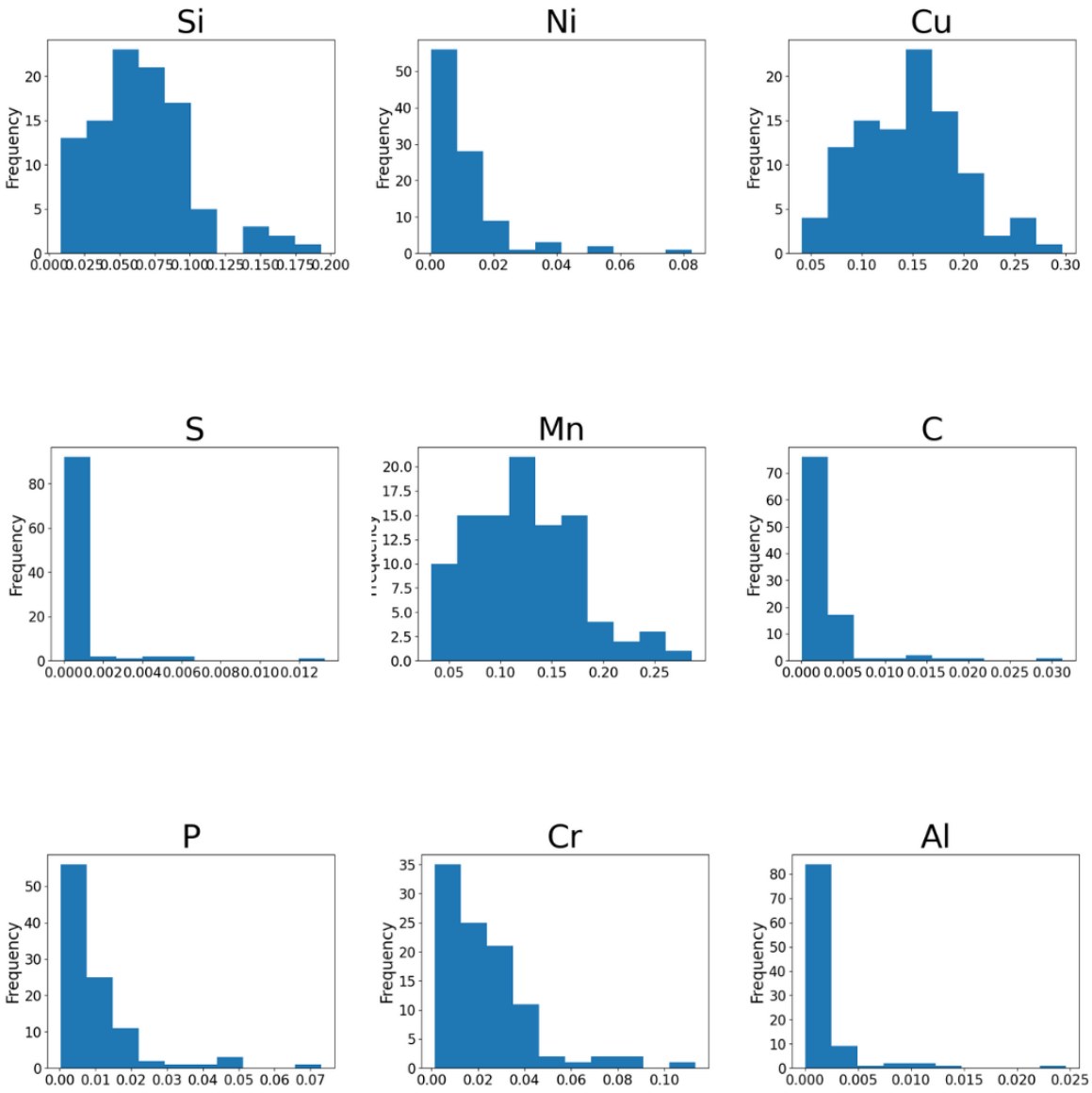


Figure 15: Distribution of each element amount of augmented St 37 by GAN

3.2 Wasserstein GAN

Observing the issues in augmenting steel data with a traditional GAN, it makes sense to implement a better and more robust method for augmentation. The mode collapse within the GAN was the limiting factor for the augmentation. Wasserstein GANs are less effected by mode collapse due to Lipschitz continuity and the use of Wasserstein distance. Gradient penalty enhances the performance and stability of the WGAN, but also increases training time drastically. Since performance and time efficiency are relevant, we implement both variations of WGANs to examine which form would be more suited for steel classification.

3.2.1 Implementation

To ensure good results, we adjust the complexity compared to the standard GAN. We increase the number of neurons in each hidden layer to 256 and 512. Furthermore, for each layer we add an activation layer with leaky ReLU, followed by batch normalization. Since the performance of the discriminator was the limiting factor in the standard GAN, we implement another hidden layer with 128 neurons. Additionally, we remove the sigmoid activation function in the output layer of the critic. The reason behind it is, that in a WGAN the critic is supposed to return a value indicating the degree to which the input is realistic, not a limited value for classification. Furthermore, the critic as well as the generator change their loss function to the Wasserstein loss. Like the paper on WGANs suggests, we use RMSProp with a learning rate of 0.0005 as the optimizers for both networks. Within the training, the critic is updated five times for each generator update. This ensures that the critic is not left behind in training. Unlike in the standard GAN, we do not add multiple instances of previously misclassified samples to the training set of the WGAN, to avoid overfitting.
As depicted in Figure 16, after 1000 epochs the critic converges and oscillates around 0. This indicates that the critic cannot differentiate between real and fake samples. On the other hand, the generator does not clearly converge, since there are still jumps towards the end of the training process. This raises the concern that the critic might have converged too rapidly, thereby not giving the best possible feedback to the generator. This could lead to an incomplete training of the generator and unrealistic sample augmentation.
Further investigation is needed, whereby we examine augmented data to gain a better picture of the performance of the generator.

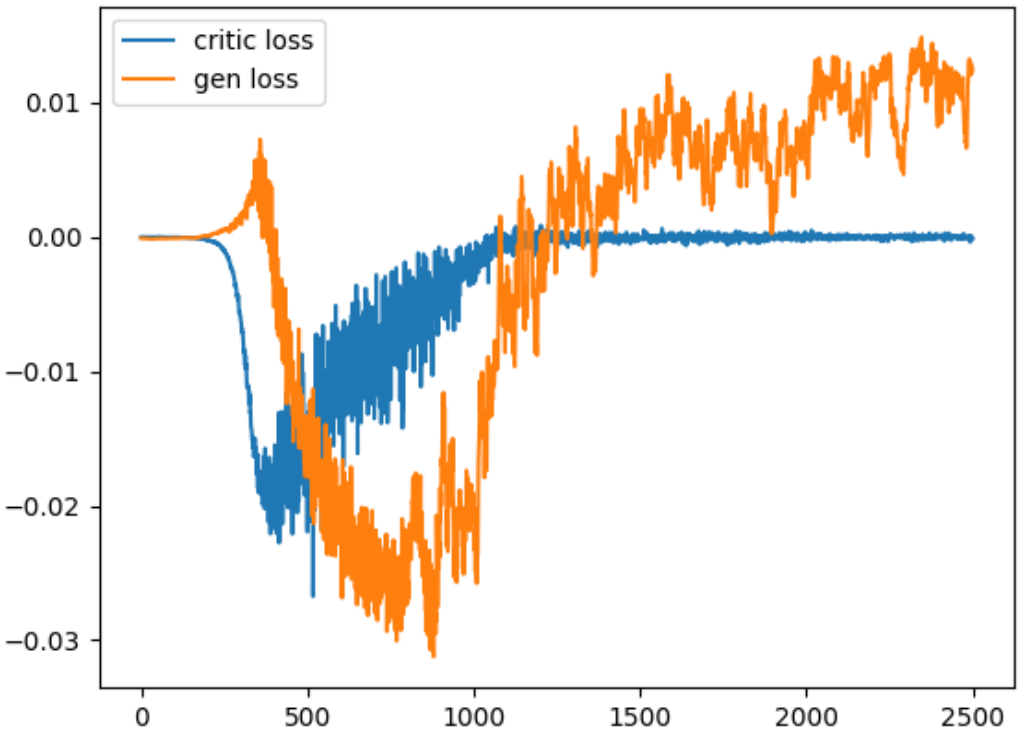


Figure 16: Loss curve WGAN St 37

3.2.2 Evaluation of Augmented Data

The implemented WGAN manages to learn the distribution of St 37 better than the traditional GAN. We reduce the dimension of the data and plot it in a 3D coordinate system for visual comparison. Figure 17 shows that the newly augmented data points resemble the original data well. The new data points are widely spread within the

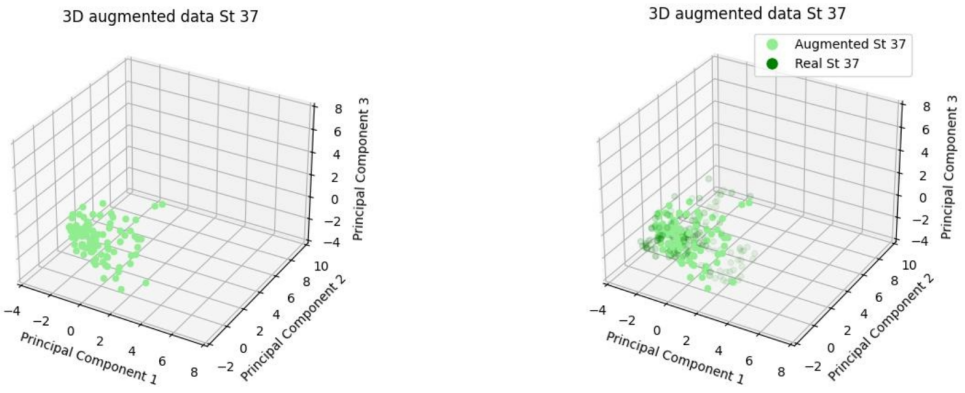


Figure 17: Augmented St 37 samples by WGAN

cluster of the original St 37 samples and capture the true variance of real data well. There is only a slight bias towards a lower value for the Principal Component 1, which is why points with a higher value are not represented well in augmented data.
To gain a better judgment of the newly augmented samples, we predict the steel type with the help of XGBoost and our Neural Network for classification. The resulting predictions are depicted in Figure 18. As expected, most data points are classified as St 37.
For both algorithms, around 20 samples are classified differently. As shown in Section 1, both algorithms have difficulties differentiating data at cluster intersections. The misclassified samples of the augmented data can be such edge cases, thereby potentially being useful for a new training set to help to differentiate.
Some data points are classified as puddling steel. Since both algorithms did not struggle to classify that steel type, these samples are ejected and have no relevance for this case.

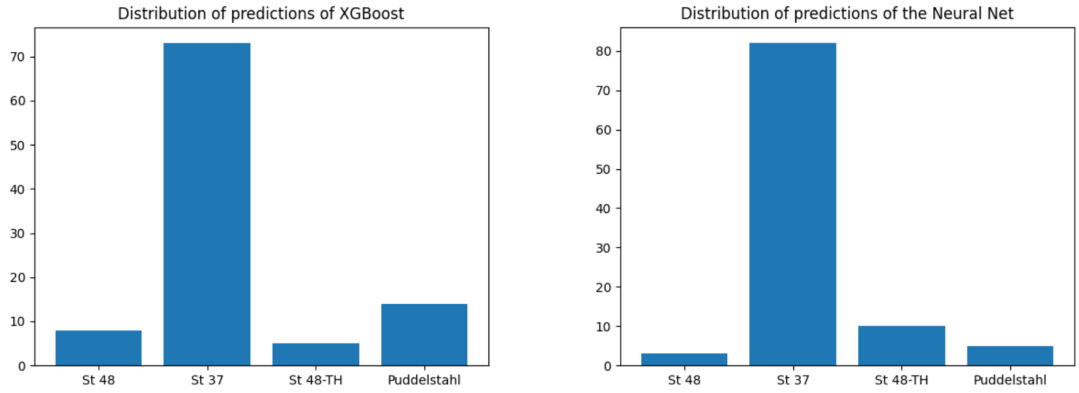


Figure 18: Distribution of class predictions of Augmented St 37

To examine the learning of the WGAN closer, we can inspect the distribution of the amount of each element taken from the augmented data, as depicted in Figure 19. The WGAN manages to recreate the original distributions excellently with the exception of minor outliers for some elements.
The overall performance is satisfactory and fulfills the expectations. The WGAN solves the previous problem of mode collapse, representing a versatile new dataset of St 37. The variance is widely spread with a small bias, as previously explained.

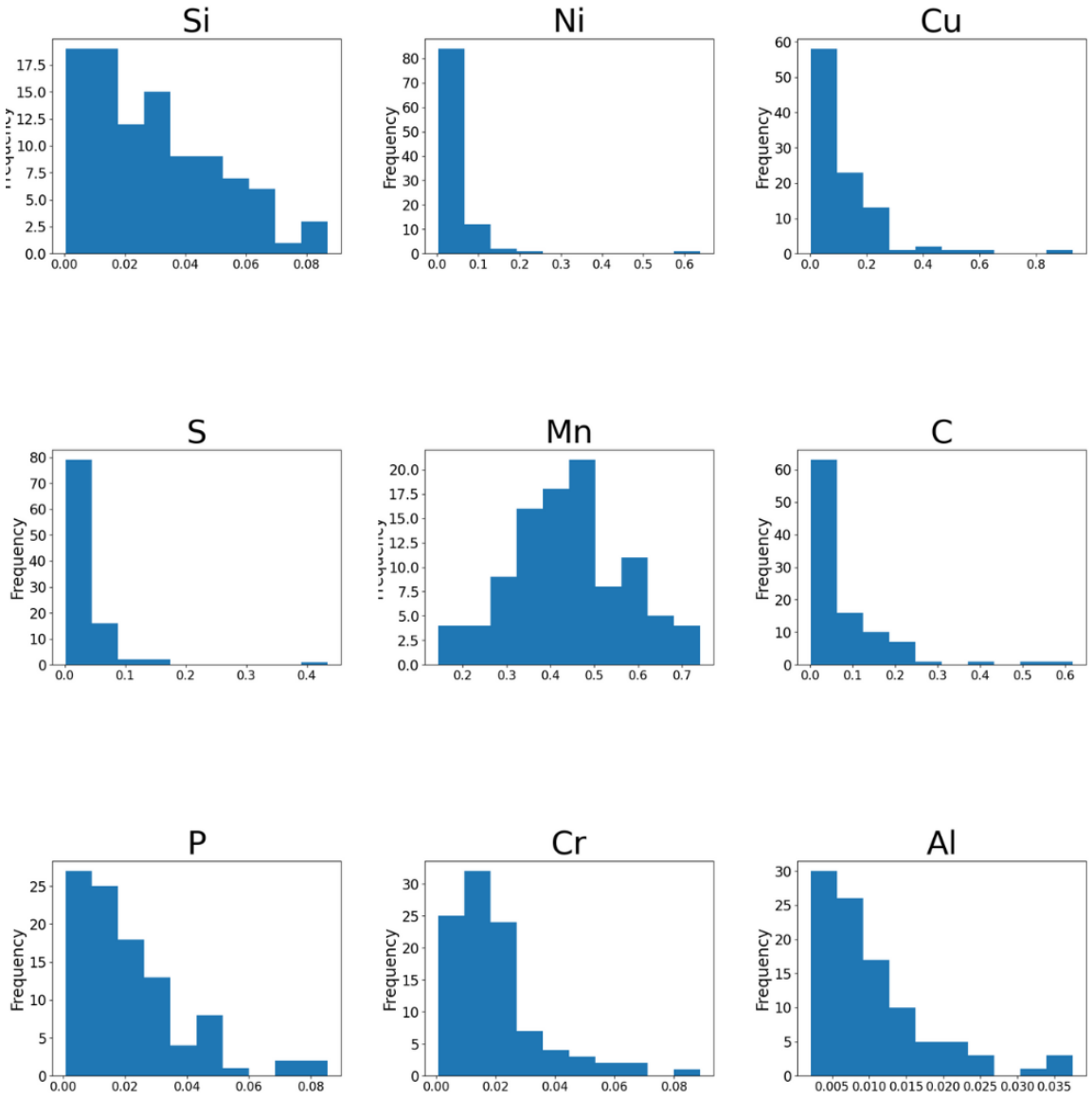


Figure 19: Distribution of each element amount of augmented St 37 by WGAN

3.2.3 Implementation of Gradient Penalty

The introduction of gradient penalty can help to stabilize the training process by enforcing 1-Lipschitz Continuity, thereby preventing the critic from saturating too quickly and improving the generator's training process. With the implementation of a gradient penalty into the WGAN, we hope to enhance the quality and realism of data even further.

We adjust the WGAN by exchanging weight clipping with a gradient penalty. The gradient penalty method creates an interpolation of each sample in the batch as described in Section 2.3.2. Subsequently, it calculates the norm of the gradient with respect to the interpolation, which will be multiplied with the regularization parameter. In the implementation, we set the regularization parameter lambda to 10, as suggested in the paper[GAA+17]. Additionally, we change the optimizers to the Adam optimizer.

Due to increased computational complexity triggered by the gradient penalty, the training of the WGAN with a gradient penalty is notably slower. Due to the time inefficiency as a consequence of limited computing power, we train the network for 1000 epochs.

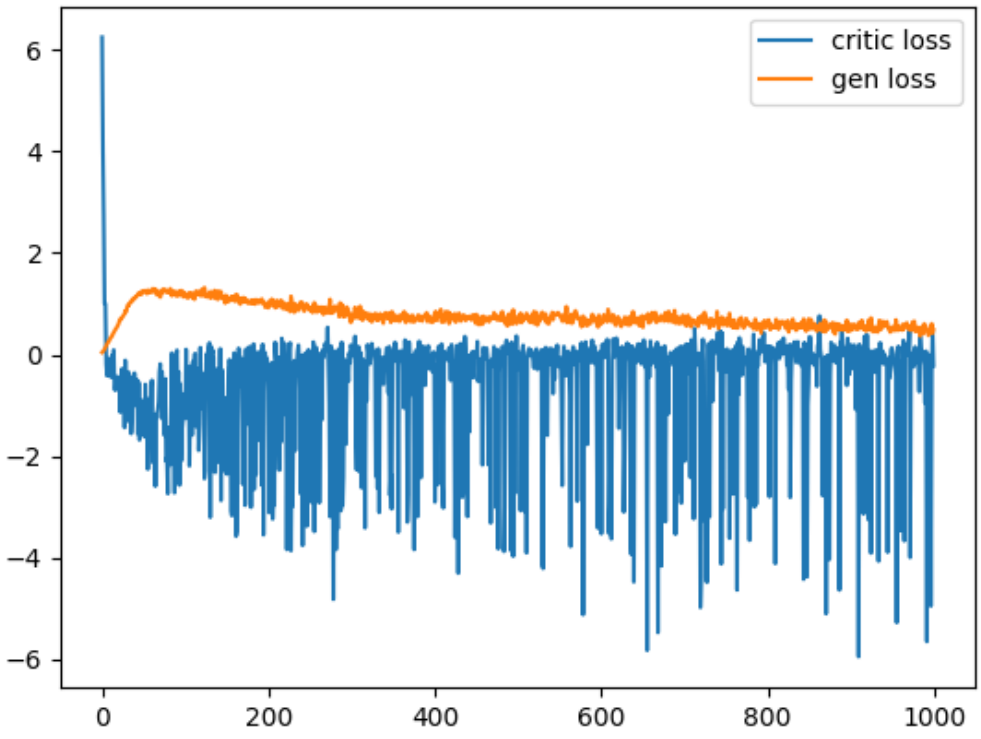


Figure 20: Loss curve WGAN-GP

As depicted in Figure 20, the generator is converging while the critic exhibits significant fluctuations due to the gradient penalty. Theoretically, the gradient penalty ensures a smooth training process. In practice for our steel data augmentation it results in instability, thereby also affecting the quality of augmented samples.

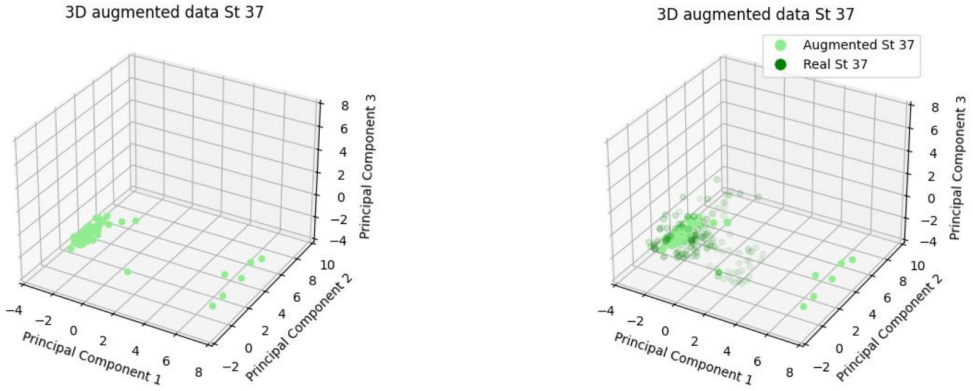


Figure 21: Augmented St 37 samples by WGAN-GP

Figure 21 shows that the augmented samples do not cover the full variance of St 37 samples. The group of outliers with a value for the Principal Component 1 of over 6 shows the poor performance of the augmentation capabilities. Additionally, augmented samples within the cluster of true St 37 are extremely similar to each other, indicating mode collapse. Due to performance issues, it does not make sense to investigate the augmented data any further.

The overall performance is worse compared to the previously introduced WGAN with weight clipping. Since the effectiveness of the gradient penalty can also depend on the specific application, we conclude that weight clipping is more suited for the case of steel type augmentation. With further hyperparameter tuning, better augmentation might be achieved, but it would not be significantly better than the WGAN with weight clipping. Besides, time-inefficiency limits the use for various cases.

3.2.4 Augmentation of St 48-TH with WGAN

The architecture of the WGAN used for augmenting St 37, can be applied to every other steel type, such as our other minority class St 48-TH. To ensure proper training and converging loss curves for the generator and critic, we will train the WGAN for 2500 epochs (Figure 22).

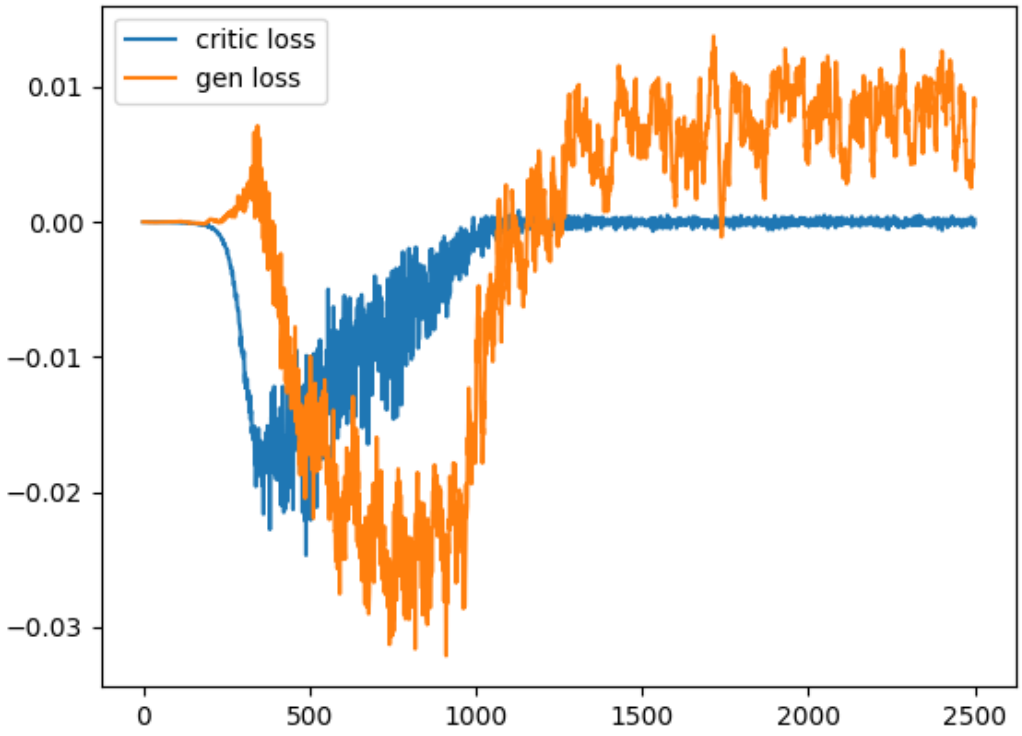


Figure 22: Loss curve WGAN St 48-TH

The newly augmented St 48-TH samples show diversity and good quality, with few outliers (Figure 23). The outliers can be removed before implementing augmented data to the new dataset.
The distributions of the predicted type, depicted in Figure 24, show a general trend towards St 48-TH, but approximately 50% of augmented samples are not classified as such. While some edge cases can be helpful for distinguishing steel types at cluster intersections, a large number of ambiguous samples leads to worse performance of supervised classification. To fix this, we limit the amount of ambiguous samples introduced in the dataset.
The number of element distributions of augmented data (Figure 25) show similar characteristics compared to the original data. The overall trend in the element amounts is continued, suggesting realistic and good-quality samples.

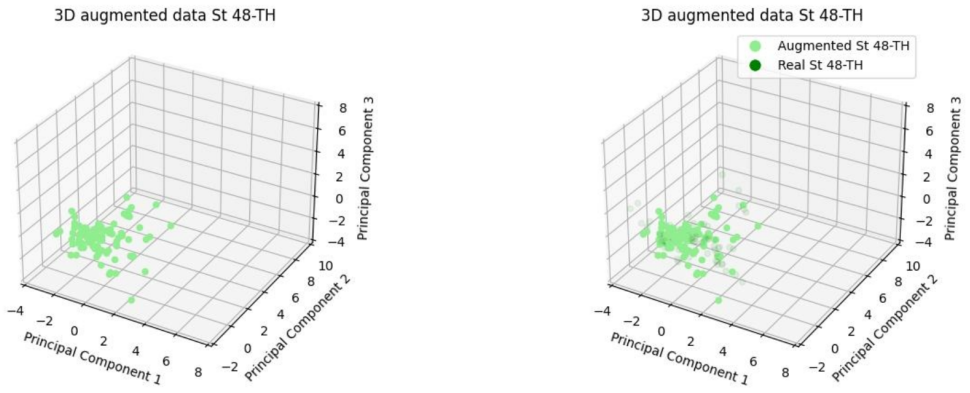


Figure 23: Augmented St 48-TH samples by WGAN

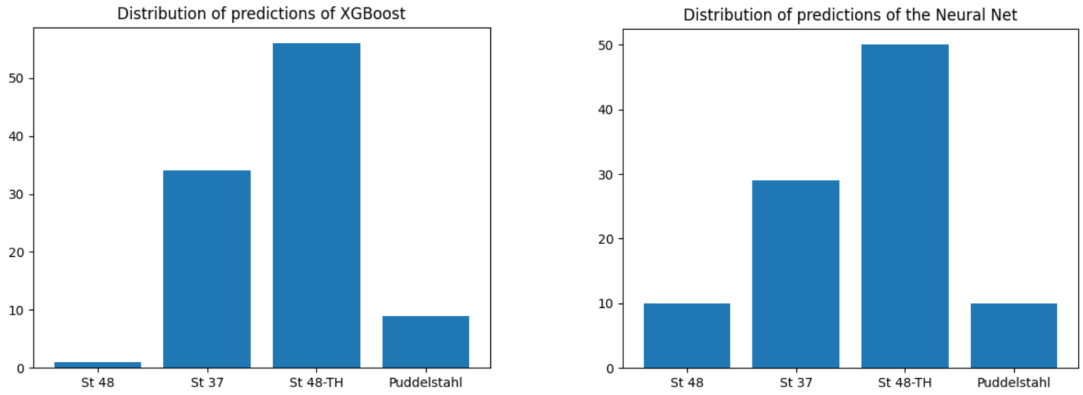


Figure 24: Distribution of class predictions of augmented St 48-TH

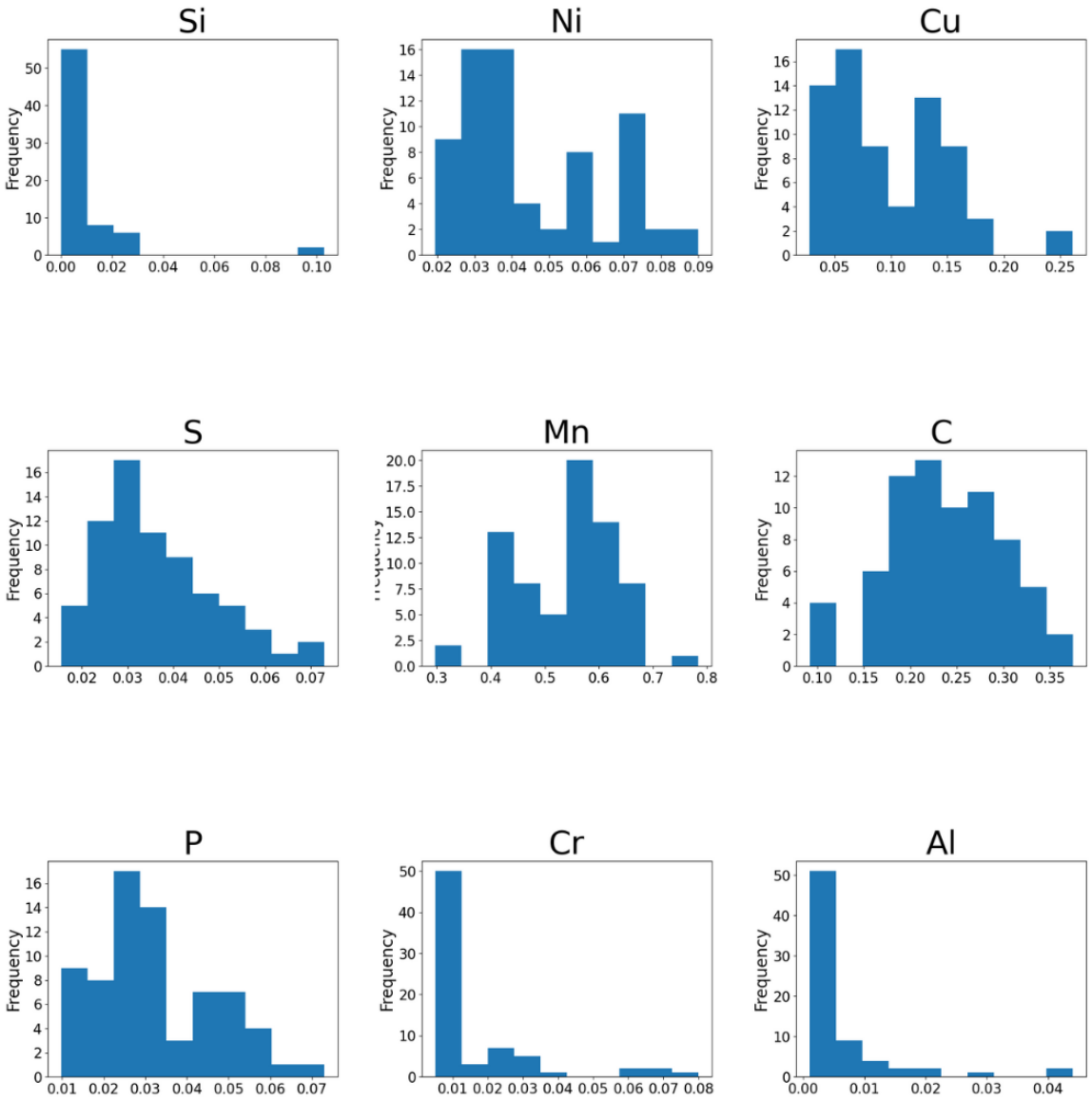


Figure 25: Distribution of each element amount of St 48-TH

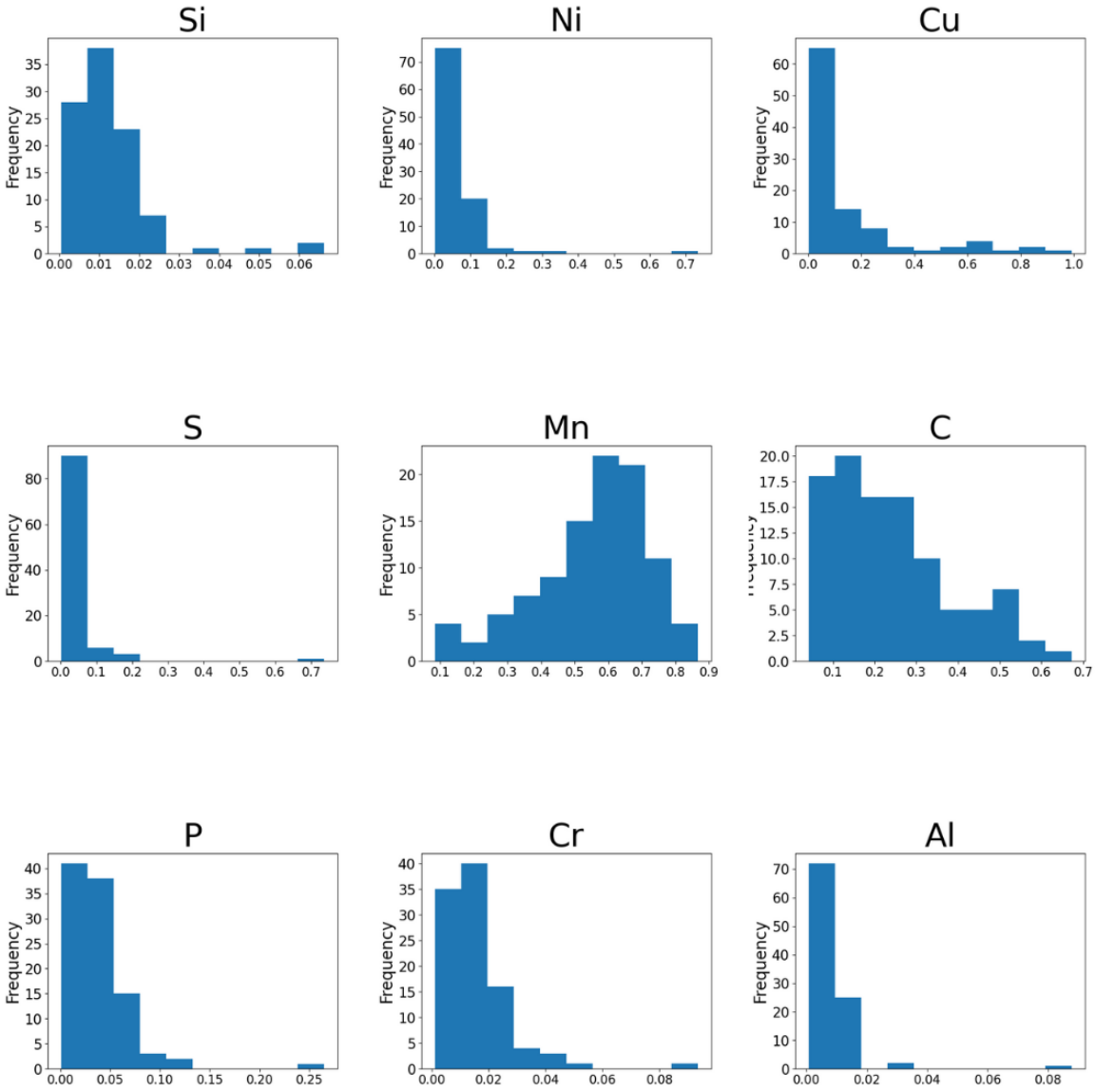


Figure 26: Distribution of each element amount of augmented St 48-TH by WGAN

3.3 SMOTE-ENN

Building a WGAN for every steel type can be time- and resource-consuming. For real-life applications, results are needed in a fast and reliable manner. To ensure a faster solution for Data Augmentation with an acceptable quality of augmented data, we implement SMOTE-ENN. This Data Augmentation technique is not capable of augmenting data for singular classes, nevertheless it is also reasonable to test this technique for this problem case, to see if it can still enhance the performance.

3.3.1 Implementation and Evaluation of Augmented Data

Due to the implementation of SMOTE-ENN in the library imblanced-learn [LNA17], the integration of our steel data is straight forward. We load the data, separate features and labels and split the data into a training and testing set. After this data preparation, we apply SMOTE-ENN, thereby oversampling the minority classes and then cleaning the data.
To evaluate the newly created dataset, we compare the distributions of steel types of the original data to the resampled data.

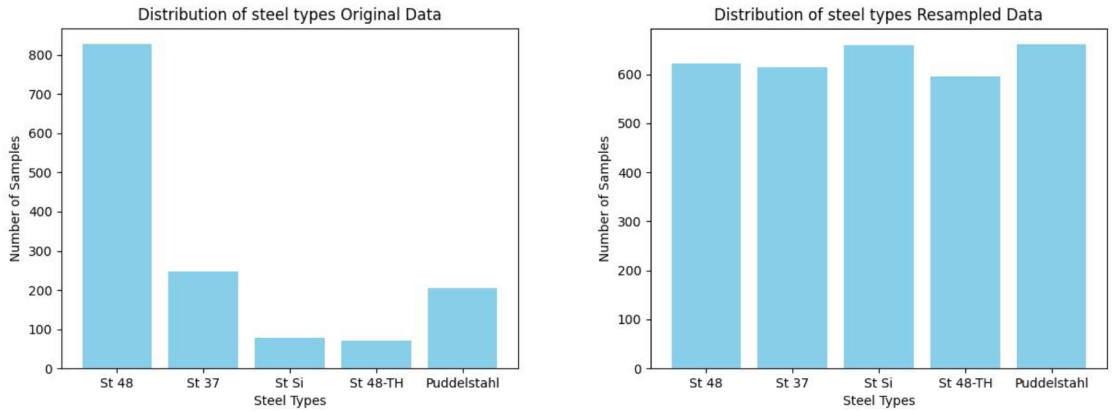


Figure 27: Distribution of steel types before and after ENN

Figure 27 shows the success of the implementation. Minority classes were balanced, whilst majority classes were undersampled. To gain a better idea on the new dataset, we visualize original data and resamples data side by side in a 3D coordinate system with the help of PCA (Figure 28).

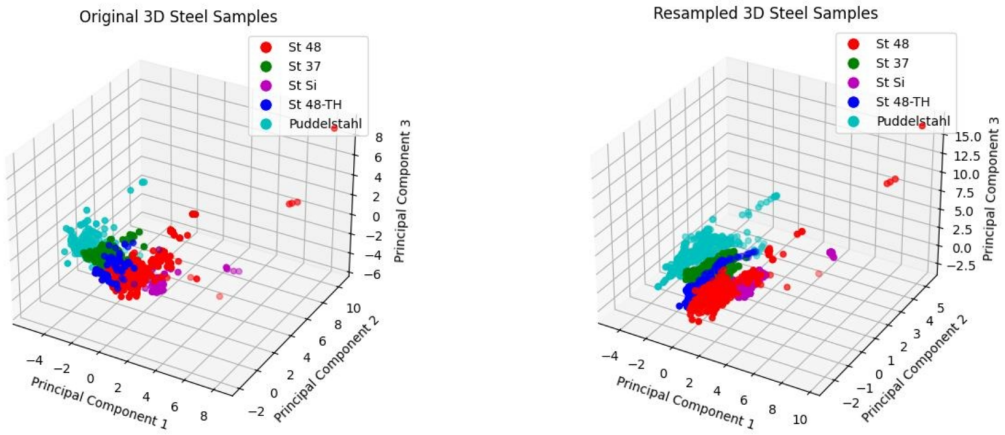


Figure 28: Original data compared to resampled data with SMOTE-ENN

In the resampled data, each cluster of steel is separated nearly without overlapping samples. In comparison to the complexity of the original data, the question arises if SMOTE-ENN oversimplifies the data. To derive a clear conclusion, we investigate the influence of the resampled data on the capabilities of both supervised classification models.

3.4 Influence of augmented Data on supervised learning

WGANs, as well as SMOTE-ENN successfully augment new data points. To see if they help to enhance the classification abilities for singular, poor performing classes, we integrate newly augmented and resampled data into the training process of XGBoost and the Neural Network for classification.

3.4.1 Augmented Data with WGAN

We augmented 100 samples of St 37, as well as St 48-TH. To balance the dataset even further, we delete 300 samples of the majority class St 48. Since both algorithms did not have difficulties classifying the minority classes puddling steel and silicon steel, we do not augment further samples.

XGBoost: The accuracy with cross validation of XGBoost declines to 91% when including the augmented data samples in the training process. With further analysis, it becomes apparent that the lower accuracy does not necessarily mean that the new model performs worse. The confusion matrix, depicted in Figure 31 (a), helps to understand the reason behind it.
The total number of misclassifications increased slightly from 13 to 14, which is not a significant change. Furthermore, the number of St 48 within the testing set is reduced from 166 to 118, due to the downsampling of the steel class. Originally, 164 of 166 samples were correctly classified. Due to the lower number of St 48 samples that can be classified correctly, classifications of other steel classes have more influence on the accuracy than before. This explains why the accuracy can be lower, but the performance did not decrease.
Due to the integration of augmented St 37 and St 48-TH samples, the previous flawlessly classified minority class silicon steel is now misclassified three times, which is also influencing the decrease of accuracy. Even though the performance for silicon steel decreased, we achieved the goal of increasing the performance for St 37 and St 48-TH (Figure 29). The f1-score increased from 0.89 to 0.91 for St 37 and from 0.73 to 0.83 for St 48-TH.
To make a direct comparison, we take the previously misclassified samples out of the training set for the new classifier and examine the performance. From the 13 previously misclassified samples, the improved classifier manages to classify five out of 13 correctly.

Figure 29: Classification report XGBoost after augmentation with WGAN

	precision	recall	f1-score	support
St 48	0.98	0.99	0.98	166
St 37	0.90	0.88	0.89	52
St Si	1.00	1.00	1.00	15
St 48-TH	0.73	0.73	0.73	15
Puddelstahl	1.00	0.97	0.99	38
accuracy			0.95	286
macro avg	0.92	0.92	0.92	286
weighted avg	0.95	0.95	0.95	286

(a) Before augmentation

	precision	recall	f1-score	support
St 48	0.97	1.00	0.98	118
St 37	0.96	0.86	0.91	63
St Si	0.84	1.00	0.91	16
St 48-TH	0.81	0.85	0.83	26
Puddelstahl	1.00	0.97	0.99	35
accuracy			0.95	258
macro avg	0.92	0.93	0.92	258
weighted avg	0.95	0.95	0.95	258

(b) After augmentation

Neural Network: The evaluation measures for the Neural Network show similar characteristics as for XGBoost. The accuracy declined to 93%, which can also be explained by the lower number of St 48 samples in the testing set. Unlike for XGBoost, the performance of the overall network did not increase with the integration of augmented data. Whilst the new classifier model increases the f1-score for St 48-TH from 0.75 to 0.82, all other f1-socres drop between 0.01 and 0.12 (Figure 30). The confusion matrix in Figure 31 (b) shows that misclassifications have shifted slightly but still appear similarly to the original model. Even though the overall performance did not increase, the new model manages to classify seven out of 11 previously misclassified samples of the original model correctly. This indicates that the new model might be more capable differentiating St 37 and St 48-TH samples, but without further unseen real data it is difficult to determine, which of the neural networks would perform better.

Figure 30: Classification report Neural Network after augmentation with WGAN

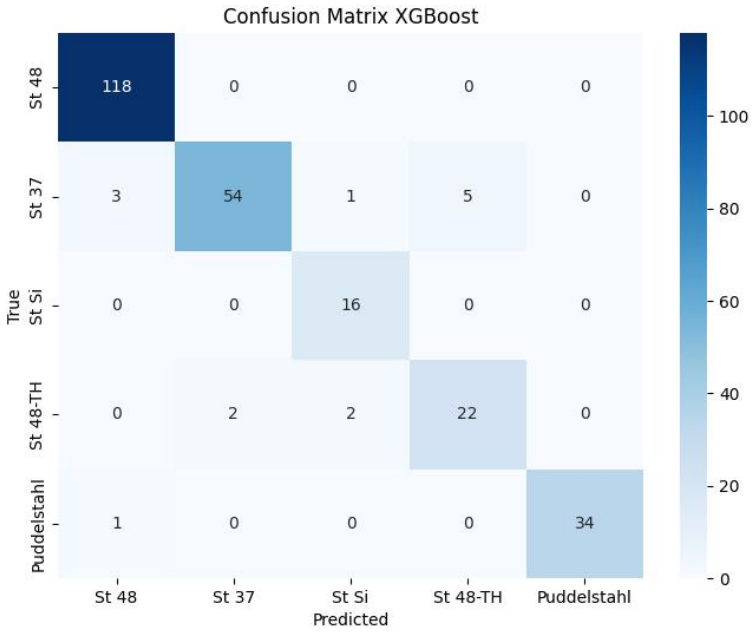
	precision	recall	f1-score	support
St 48	0.98	0.99	0.98	166
St 37	0.94	0.88	0.91	52
St Si	1.00	1.00	1.00	15
St 48-TH	0.71	0.80	0.75	15
Puddelstahl	1.00	1.00	1.00	38
accuracy			0.96	286
macro avg	0.93	0.93	0.93	286
weighted avg	0.96	0.96	0.96	286

(a) Before augmentation

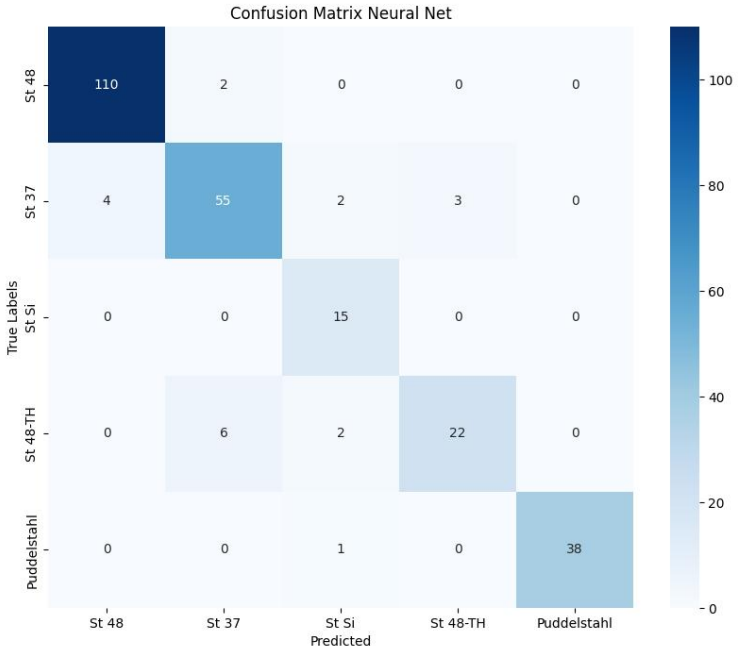
	precision	recall	f1-score	support
St 48	0.96	0.97	0.97	112
St 37	0.86	0.86	0.86	64
St Si	0.79	1.00	0.88	15
St 48-TH	0.88	0.73	0.80	30
Puddelstahl	1.00	1.00	1.00	39
accuracy			0.92	260
macro avg	0.90	0.91	0.90	260
weighted avg	0.92	0.92	0.92	260

(b) After augmentation

Figure 31: Evaluation of augmented data with WGAN



(a) Confusion Matrix of XGBoost with augmented data



(b) Confusion Matrix of Neural Network with augmented data

3.4.2 Augmented Data with SMOTE-ENN

After the application of SMOTE-ENN, it is crucial to set the testing data correctly. If the test data for the newly trained model is taken from the resampled data, the resulting performance is not representative. Due to the fact that noisy and ambiguous data are removed from the dataset by SMOTE-ENN, classification on those samples will be flawless, which is why it is important to ensure the involvement of realistic data for testing.

XGBoost: With the integration of resampled data produced by SMOTE-ENN, the accuracy of XGBoost reaches 93%, with the f1-scores of St 37 and St 48-TH slightly dropping, indicating a worse performance on those steel classes. The confusion matrix for the model, depicted in Figure 34 (a) does not exhibit significant changes compared to the provided model. Furthermore, the classification report depicted in Figure 32 shows a slight drop of performance for St 37 and St 48-TH. Since this counteracts the goal of improving classification for both classes, in this case resampling with SMOTE-ENN is useless.
To see if the classification abilities of the model changed, we use the new classifier on 13 data samples that were previously misclassified by the original XGBoost model. Out of 13 samples, the new model classifies five correctly. While the model seems to improve classifying those samples, the overall classification abilities did not improve.

Figure 32: Classification report XGBoost after resampling with SMOTE-ENN

	precision	recall	f1-score	support
St 48	0.98	0.99	0.98	166
St 37	0.90	0.88	0.89	52
St Si	1.00	1.00	1.00	15
St 48-TH	0.73	0.73	0.73	15
Puddelstahl	1.00	0.97	0.99	38
accuracy			0.95	286
macro avg	0.92	0.92	0.92	286
weighted avg	0.95	0.95	0.95	286

(a) Before augmentation

	precision	recall	f1-score	support
St 48	1.00	0.93	0.96	166
St 37	0.78	0.88	0.83	52
St Si	1.00	1.00	1.00	15
St 48-TH	0.63	0.80	0.71	15
Puddelstahl	0.97	1.00	0.99	38
accuracy			0.93	286
macro avg	0.88	0.92	0.90	286
weighted avg	0.94	0.93	0.93	286

(b) After augmentation

Neural Network: The Neural Network trained on resampled data of SMOTE-ENN reaches an accuracy of 93%. Similarly to the XGBoost model, the performance did not improve. As depicted in Figure 34 (b), the amount of misclassifications is similar to the original model. Furthermore, the classification report in Figure 33 shows a drop of performance for St 37 and St 48-TH. The new model did not manage to enhance the classification.
The neural network manages to classify three samples correctly among the misclassified samples by the original model, which suggest slight changes in the classification abilities.

Figure 33: Classification report XGBoost after resampling with SMOTE-ENN

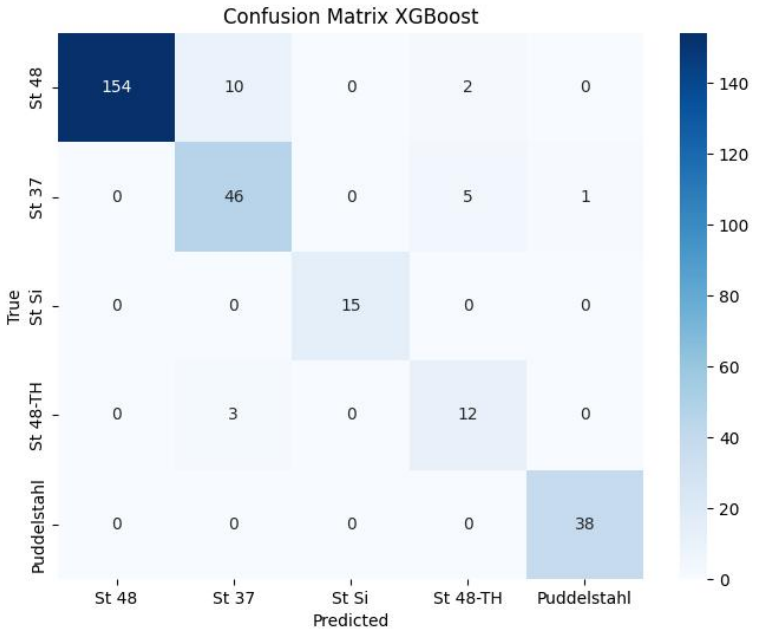
	precision	recall	f1-score	support
St 48	0.98	0.99	0.98	166
St 37	0.94	0.88	0.91	52
St Si	1.00	1.00	1.00	15
St 48-TH	0.71	0.80	0.75	15
Puddelstahl	1.00	1.00	1.00	38
accuracy			0.96	286
macro avg	0.93	0.93	0.93	286
weighted avg	0.96	0.96	0.96	286

(a) Before augmentation

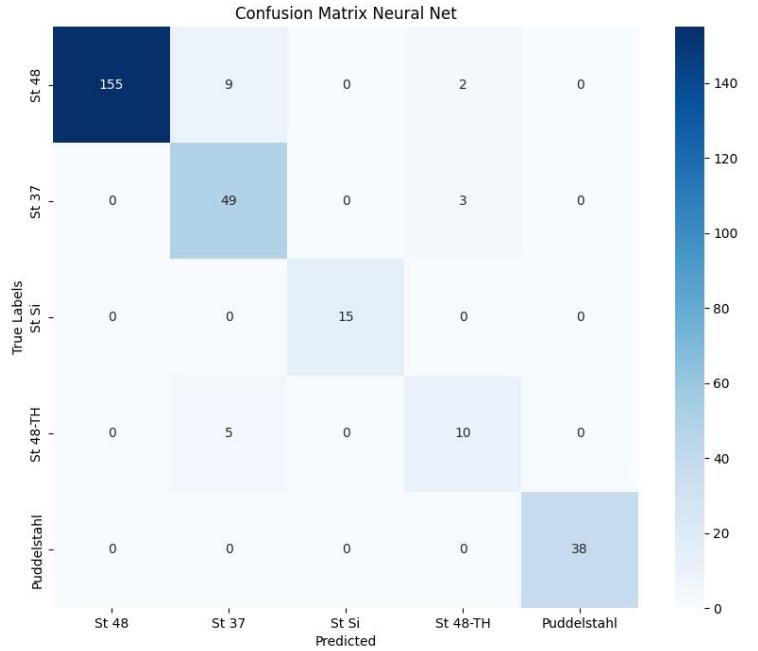
	precision	recall	f1-score	support
St 48	1.00	0.93	0.97	166
St 37	0.78	0.94	0.85	52
St Si	1.00	1.00	1.00	15
St 48-TH	0.67	0.67	0.67	15
Puddelstahl	1.00	1.00	1.00	38
accuracy			0.93	286
macro avg	0.89	0.91	0.90	286
weighted avg	0.94	0.93	0.94	286

(b) After augmentation

Figure 34: Evaluation of augmented data with SMOTE-ENN



(a) Confusion Matrix of XGBoost with augmented data



(b) Confusion Matrix of Neural Network with augmented data

3.5 Conclusion

We successfully enhanced the original dataset with the presented Data Augmentation techniques.
While the traditional GAN manages to augment data, mode collapse is a crucial limiting factor that leads to a small variance in augmented data. Since realistic, high-quality samples are essential, this technique is not integrated in the program.
Wasserstein GANs represent a more robust augmentation technique to traditional GANs and are less prone to mode collapse while producing realistic data. To enhance the functionality and robustness even further, a gradient penalty can be implemented. Especially due to the time inefficiency because of extra calculations for the penalty value, we deem the functionality not to be suited for our case.
SMOTE-ENN offers an approach that over-samples minority classes before cleaning noisy and ambiguous data. The resulting data is almost flawlessly balanced.
The application of the WGANs and SMOTE-ENN to the dataset provided shows changes in the classification abilities of supervised learning models.
With the integration of augmented St 37 and St 48-TH samples from WGANs in the training process of the XGBoost classifier, we enhanced the f1-score for both steel types, which were frequently misclassified by the original classifiers. The neural network for classification slightly decreased its accuracy overall, while increasing the performance on St 48-TH. The neural network managed to classify seven out of 11 previously misclassified St 48-TH and St 37 samples correctly, indicating an increased performance in differentiating St 37 and St 48-TH overall. To determine if both new models extensively differ from the original models, further evaluation is needed with new unseen data.
SMOTE-ENN successfully balances the whole dataset with over- and under-sampling. For this particular problem case of singular poor performing steel classes, the Data Augmentation technique did not show any improvements on supervised learning and even counteracted the goal. With this technique, it is not possible to target single steel classes to improve their classification. SMOTE-ENN performs best for use cases with generally few data points (Section 4).
By extracting all data samples previously misclassified by both original supervised learning models and excluding them from the training process of the new classifiers, the overall change on classification can be approximated. Passing the data samples through the new classifiers trained on augmented data of SMOTE-ENN and WGANs, all models manage to classify between 27%-64% samples correctly, indicating a change of the general classifications. WGANs fulfill necessary requirements for the use in the steel classification software. Singular steel classes can be augmented realistically and enhance the dataset. The consequences of augmented samples have to be examined extensively, since an overall improvement of supervised learning is not guaranteed.
For the integration of data augmentation for singular classes with WGANs into the steel classification software, we implement a method that trains a WGAN for the given steel class. The user can set the desired amount of augmented data samples, which we save to a csv file. This data can then be processed individually for each use case. Hyperparameters required to train the WGAN are hardcoded with values that worked for the examined cases to make the application possible without background knowledge about this technique.

4 Application of Data Augmentation on Overall Imbalanced Data

4.1 Exemplary dataset

To show a useful case, where both Data Augmentation techniques can be used, we form a new dataset from the original data. The dataset comprises 120 data points with 60 St 48-TH samples, 20 St 37 and puddling steel samples, and ten St 48 and silicon steel samples (Figure 35). In this case, St 48-TH is the majority class with all other classes as minority classes in comparison.

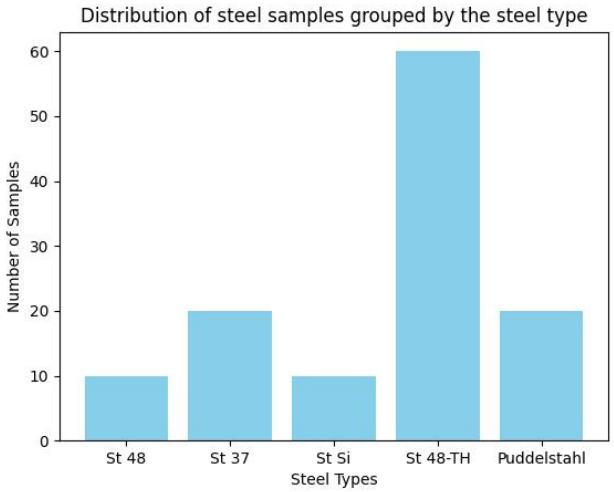


Figure 35: Distribution of steel classes in application dataset

To determine the performance of both supervised learning models, we firstly train both on the imbalanced dataset. Subsequently, we use the unseen data from the original dataset to test the classification abilities. For evaluation, we focus primarily on f1-scores and confusion matrices, since both methods provide clear, interpretable values.

4.2 Influence of Data Augmentation on supervised learning

XGBoost

The initial classification report shows good performance of the trained XGBoost model on St 48 and silicon steel (Figure 36 (a)). Even though the training data only contained 10 samples of both steel classes, the performance is decent. The low f1-score of 0.15 for St 48-TH is especially striking at first glance, since it is the only majority class in the training data. A contributing factor for this is the fact that the testing data only contains 11 samples of St 48-TH. As depicted in Figure 37 (a), all 11 true St 48-TH samples have been correctly classified, but 67 St 48 and 61 St 37 samples have been

incorrectly classified as St 48-TH, resulting in a small precision and thereby strongly reducing the f1-score. This indicates that the model has a bias towards St 48-TH, induced by the imbalanced training data. Overall, the model reaches an accuracy of 83%. With the use of our Data Augmentation techniques, we hope to increase the overall performance by enhancing the capabilities of classifying steel types.

Augmentation with WGANs: For augmentation, we set the minimal value of samples per class to 300. Training a new XGBoost model on the enhanced dataset increases the performance significantly, as depicted in Figure 36 (b). For St 48, St 37, St 48-TH and puddling steel, the f1-scores increase between 0.06 and 0.16, indicating improvements for the functionality. Despite fewer St 48 and St 37 samples being misclassified as St 48-TH, more than half of the actual St 48-TH samples are now incorrectly classified as St 37, indicating difficulties differentiating between St 37 and St 48-TH (Figure 37 (b)). Only the performance of silicon steel decreases slightly by 0.06. Even though the model manages to classify each real silicon steel correctly, the number of samples incorrectly classified as silicon steel increases.
After using WGANs to augment new data, the overall performance improved, leading to an increase of the classifying accuracy from 83% to 94%.

Resampling with SMOTE-ENN: The resampling of the dataset with SMOTE-ENN shows similar effects on the model as the augmentation with WGANs. The f1-scores for St 48, St 37 and puddling steel increase around 0.05 to 0.12 (Figure 36 (c)). While the classification abilities of silicon steel remain exactly the same, the performance on St 48-TH decreases even further. The low value for recall as well as precision indicates that the new model loses the ability to differentiate between St 48-TH and other steel types. As presented in Figure 37 (c), only two of eleven St 48-TH-samples are classified correctly. Even though the overall performance increases to 92%, it is questionable if the loss of the functionality to classify St 48-TH is justifiable, as this could impact the system's ability to handle specific cases where St 48-TH classification is critical.

Neural Network

The Neural Network reaches an accuracy of 81% on the imbalanced training data. While it classifies St 48 and puddling steel well with f1-scores of 0.91 and 0.89, the model performs worse on all other steel classes (Figure 38 (a)). For St 48-TH, St 37 and silicon steel, the f1-scores range from 0.14 to 0.68. Investigating the confusion matrix in Figure 39 (a), it becomes apparent that the network has difficulties recognizing true St 37 samples. Almost 50% of St 37 samples in the testing data were incorrectly classified. Furthermore, for St 48 the model misclassifies true steel samples as silicon steel and St 48-TH. Even though all 11 samples of St 48-TH are classified correctly, a large number of St 48 and St 37 samples are also classified as St 48-TH. Similarly to the initial XGBoost model, we observe a bias towards St 48-TH.

Augmentation with WGANs: With the incorporation of augmented data by the generators of WGANs, the performance of the model enhances noticeably. The classes St 48 and puddling steel, which were already classified well in the original training set, increase their f1-score to 0.96 and 0.97 (Figure 38 (b)). Even more significant is the improvement of the f1-scores of St 37 and silicon steel, rising to a value of 0.87 and 0.93. Whilst the classification function for these four steel classes improves, the f1-score for St 48-TH further decreases to 0.07. The confusion matrix in Figure 39 (b) illustrates that the model loses its ability to classify St 48-TH samples by falsely classifying 10 out of 11 samples in the testing data as St 37.
Overall, the presented enhancements help to an increase the model's accuracy to reach a value of 93%.

Resampling with SMOTE-ENN: SMOTE-ENN also increases the performance, leading to an accuracy of 91%. For each steel class, the f1-scores increase between 0.04 and 0.24 (Figure 38 (c)). Despite the fact that the f1-score for St 48-TH rises by 0.04, the confusion matrix, as depicted in Figure 39 (c), does not show a clear indication of an improvement. Even though fewer St 48 and St 37 samples are classified as St 48-TH, almost half of true St 48-TH samples are now incorrectly classified.

Figure 36: Classification reports for XGBoost before and after Data Augmentation

	precision	recall	f1-score	support
St 48	0.94	0.87	0.90	817
St 37	0.77	0.69	0.73	227
St Si	0.93	0.97	0.95	69
St 48-TH	0.08	1.00	0.15	11
Puddelstahl	0.97	0.76	0.85	185
accuracy			0.83	1309
macro avg	0.74	0.86	0.72	1309
weighted avg	0.91	0.83	0.86	1309

(a) Before Data Augmentation

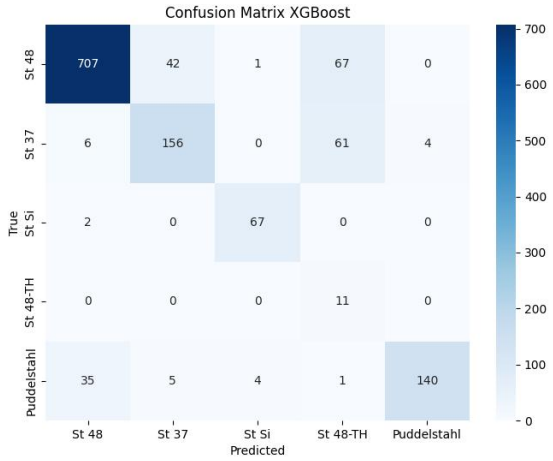
	precision	recall	f1-score	support
St 48	0.97	0.96	0.96	817
St 37	0.90	0.87	0.89	227
St Si	0.80	1.00	0.89	69
St 48-TH	0.24	0.45	0.31	11
Puddelstahl	1.00	0.96	0.98	185
accuracy			0.94	1309
macro avg	0.78	0.85	0.81	1309
weighted avg	0.95	0.94	0.94	1309

(b) After Data Augmentation with WGANs

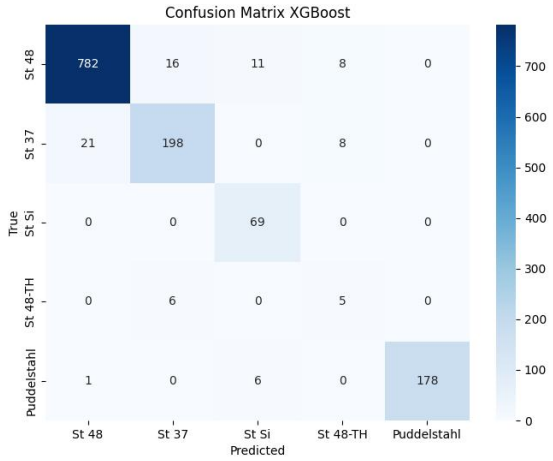
	precision	recall	f1-score	support
St 48	0.98	0.93	0.95	817
St 37	0.81	0.88	0.84	227
St Si	0.93	0.97	0.95	69
St 48-TH	0.05	0.18	0.08	11
Puddelstahl	0.99	0.95	0.97	185
accuracy			0.92	1309
macro avg	0.75	0.78	0.76	1309
weighted avg	0.94	0.92	0.93	1309

(c) After Data Augmentation with SMOTE-ENN

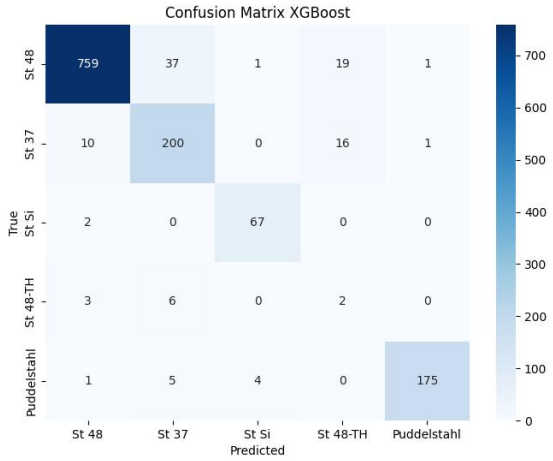
Figure 37: Confusion matrices for XGBoost before and after Data Augmentation



(a) Before Data Augmentation



(b) After Data Augmentation with WGANs



(c) After Data Augmentation with SMOTE-ENN

Figure 38: Classification reports for the Neural Network before and after Data Augmentation

	precision	recall	f1-score	support
St 48	0.97	0.86	0.91	817
St 37	0.98	0.50	0.66	227
St Si	0.51	1.00	0.68	69
St 48-TH	0.08	1.00	0.14	11
Puddelstahl	0.87	0.92	0.89	185
accuracy			0.81	1309
macro avg	0.68	0.86	0.66	1309
weighted avg	0.93	0.81	0.85	1309

(a) Before Data Augmentation

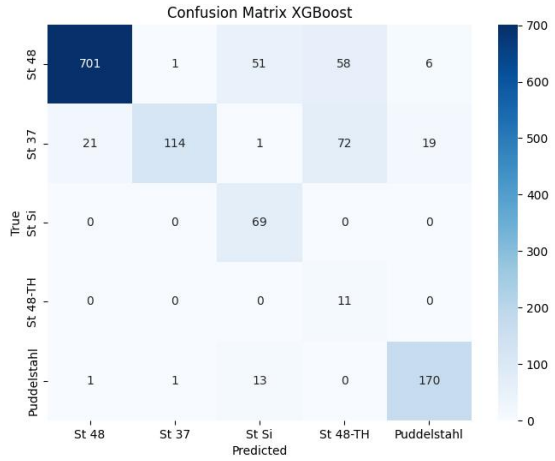
	precision	recall	f1-score	support
St 48	0.97	0.95	0.96	817
St 37	0.85	0.89	0.87	227
St Si	0.86	1.00	0.93	69
St 48-TH	0.06	0.09	0.07	11
Puddelstahl	1.00	0.94	0.97	185
accuracy			0.93	1309
macro avg	0.75	0.78	0.76	1309
weighted avg	0.94	0.93	0.94	1309

(b) After Data Augmentation with WGANs

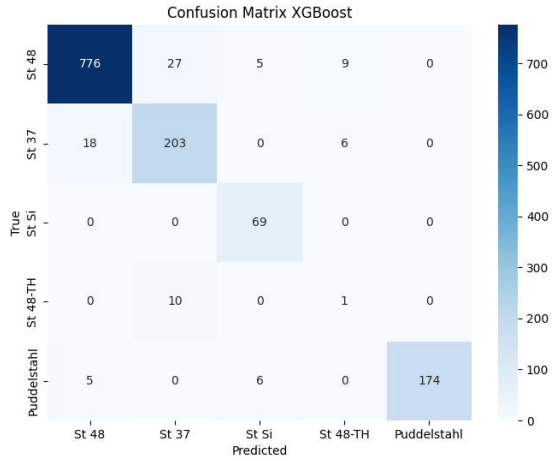
	precision	recall	f1-score	support
St 48	0.96	0.93	0.95	817
St 37	0.97	0.83	0.90	227
St Si	0.67	1.00	0.80	69
St 48-TH	0.11	0.55	0.18	11
Puddelstahl	1.00	0.94	0.97	185
accuracy			0.91	1309
macro avg	0.74	0.85	0.76	1309
weighted avg	0.95	0.91	0.93	1309

(c) After Data Augmentation with SMOTE-ENN

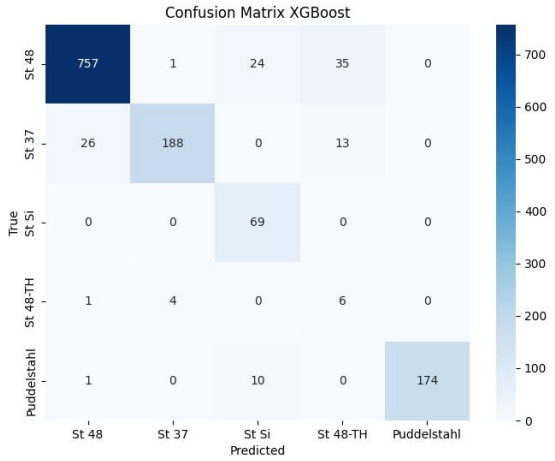
Figure 39: Confusion matrices for the neural net before and after Data Augmentation



(a) Before Data Augmentation



(b) After Data Augmentation with WGANs



(c) After Data Augmentation with SMOTE-ENN

4.3 Conclusion

The utility of WGANs and SMOTE-ENN enhanced the performance of both supervised learning algorithms significantly. The accuracy for classification on this exemplary training dataset increased between 10% - 12%. For most steel classes, the f1-scores rose consistently with the exception of St 48-TH. Despite the fact that St 48-TH was the majority class within the training data, both supervised models did not acquire the ability to classify the steel type reliably.
Depending on how critical the correct classifications of single classes are, the user of the steel classification software needs to check evaluation measures to ensure desired performance of the classification models, with and without the use of Data Augmentation techniques.

To integrate both Data Augmentation techniques into the steel classification project, we make the process independent of the number of steel classes and shapes of the input data. For an intuitive application of Data Augmentation techniques, we introduce two methods that either create WGANs and augment data or apply SMOTE-ENN on the dataset provided. Whilst SMOTE-ENN undertakes over- and under-sampling of the data automatically, the user has to set a minimal and maximal number of steel samples that will be augmented or under-sampled automatically. The user can derive both numbers from the original dataset.
The training of WGANs as well as SMOTE-ENN require numerous hyperparameters, which we set to values that proved to be working for the steel classes provided so far. Accordingly, we ensure that the application is possible without background knowledge of each Data Augmentation technique.
By utilizing SMOTE-ENN and WGANs, it is now possible to resample data to achieve fast results that can improve supervised learning, or augment realistic data that is specific for each steel class for numerous use cases, thereby also contributing to the enhancement of supervised learning.

5 Discussion

5.1 Limitations

The Data Augmentation techniques implemented in this thesis show limitations concerning their usages. In Section 3, we examined the influence of augmented data of Generative Adversarial Networks for singular classes on the performance of the supervised learning algorithms provided. Even though both models had a decent performance overall, the performance on St 48-TH and St 37 was significantly lower than for the remaining steel types. With the use of Wasserstein GANs, we managed to boost the f1-score for St 48-TH of both models, while the performance on St 37 with augmented data only increased for XGBoost. Furthermore, the classification abilities of supervised learning for some steel classes decreased. Despite the fact that the originally provided dataset is heavily imbalanced, in this case it might not be useful to apply our Data Augmentation techniques, depending on how critical the correct classification of St 48-TH is. If the correct classification of St 48-TH is more important than other steel types, this trade-off could be accepted. Otherwise, if the classification is equally essential, it might not improve the performance overall.
The consequences of the application of Data Augmentation techniques has to be investigated to ensure increased performance for the individual use case.

Both Data Augmentation techniques have further technical limitations, which are compensated for by the respective other technique.
SMOTE-ENN does not operate well on high dimensional data. Due to the curse of dimensionality, the distance measures used in the algorithm become less meaningful as the number of dimensions increases. Accurately identifying nearest neighbors of data points becomes increasingly difficult, resulting in poor synthetic data. In case of high dimensional data, GANs have been proven effective. In case of high dimensional data, GANs are the preferable Data Augmentation technique.
However, initially training a WGAN for a specific steel type can be extremely time-consuming. When dealing with numerous different steel classes, the total training time might exceed acceptable limits for users. For a fast solution, SMOTE-ENN is a reliable alternative.

5.2 Future Work

The next step will be to integrate the Data Augmentation functionality into the frontend and backend of the project to make it available for the user. In the frontend, the data could be visualized through dimensional reduction, with a direct comparison of the supervised learning algorithms before and after augmentation. Since in some cases the models trained on augmented data might lose functionalities, which are critical for the use case, the user should be able to select a model depending on which classification is preferred.

To gain an extensive idea of the performance of both Data Augmentation techniques, further investigation is needed. The provided data is limited, with only a view of numerous types of steel and little data samples for certain steel classes. Depending on the data given into a Wasserstein Generative Adversarial Network, hyperparameters might need to be changed or tuned to ensure good quality augmentation.
Hyperparameter turning in general can be continued. Due to the limited computing power, used to train the WGANs, only a limited number of hyperparameter variations were tested. With the integration of tuning aids like grid search, parameters could be chosen more efficiently.
To achieve a higher computing power, a connection to a server could be established that enables faster training of WGANs and opens the possibility of extensive hyperparameter optimization. This would also allow training WGANs with gradient penalty more extensively until they might perform better than the current WGANs.

Furthermore, additional Data Augmentation techniques, similar to SMOTE-ENN and GANs could be investigated. Variations of GANs, such as Conditional GANs [MO14], and other over- or under-sampling approaches, for example ADASYN[HBGL08] also have promising abilities that could be applied to this case.
With the incorporation of more Data Augmentation techniques, multiple methods could be combined to further balance the dataset and enhance the overall augmentation abilities to create realistic data. An ensemble of Data Augmentation techniques might help supervised learning algorithms even further to differentiate steel classes.
Besides new augmentation techniques, new supervised learning models could also be implemented to see if the impact of Data Augmentation might lead to better results compared to the integrated XGBoost and the Sequential Neural Network.
To help the augmentation of realistic data, feedback loops where the user can rate the performance could be integrated. Feedback could be stored in a database until its analyzed and utilized to improve Data Augmentation techniques by fine-tuning and retraining.

The integration of Data Augmentation will enhance the capabilities of the whole system. Continued hyperparameter tuning, leveraging higher computing power and exploring additional Data Augmentation techniques will make Data Augmentation for steel classification more applicable. These advancements will expand the range of use cases for similar problems, providing a robust foundation to create realistically augmented steel data.

5.3 Conclusion

This thesis has explored the application of Data Augmentation techniques in steel classification. From the dataset provided we derived two main problems that can influence the performance of supervised learning negatively. First, the steel dataset was heavily imbalanced. Second, the supervised learning models exhibited worse performance on singular classes. To solve these problems, we explored Data Augmentation techniques with promising abilities.
To augment data to improve the performance of singular classes, we examined Generative Adversarial Networks, Wasserstein Generative Adversarial Networks and Synthetic Minority Over-sampling Technique in combination with Edited Nearest Neighbors. The augmented data of traditional GANs did not cover the whole variance of data points within a steel class cluster, due to mode collapse. This issue led to low quality samples, which were not useful for the steel data. WGANs fixed this issue with satisfactory augmentation capabilities by recreating the underlying data distribution of steel classes. For the use cases within the steel classification software, we exclude the gradient penalty in the training process of WGANs, since it is not beneficial on the steel data provided and it increases training time substantially.
While WGANs increased the f1-scores of poor performing classes the performance on other classes decreased. Depending on how critical the classification of the poorly classified class is, this trade-off can be accepted or rejected.
SMOTE-ENN managed to resample data reliably, but did not improve the performance on singular poor performing classes. For supervised learning models that have poor performance overall due to an imbalance of the training data, WGANs and SMOTE-ENN can help to improve classification significantly. In the application example, the overall accuracy increased by 10% to 12% due to Data Augmentation. The majority of steel classes showed a consistent rise in f1-scores. This demonstrates the practicality of the application of Data Augmentation techniques for handling imbalanced datasets. While SMOTE-ENN enhanced the performance by increasing accuracy and f1-scores, the application is limited by a large number of dimensions of data, where distance measures become less meaningful.
WGANs require significant training time, making them less practical for real-time applications with numerous steel classes, despite the fact that it generates high-quality data, including in higher dimensions.
The choice of the Data Augmentation technique is dependent on the specific requirements of the classification task. The user needs to evaluate potential trade-offs concerning time efficiency and the quality of augmented data.

In conclusion, the integration of WGANs and SMOTE-ENN into the steel classification software can significantly enhance the performance of supervised learning models on heavily imbalanced datasets. Future work should focus on the presented limitations on certain steel classes. Furthermore, the training time of WGANs can be optimized to make them more efficient and broadly applicable within the software. The findings in this thesis highlight the relevance of adapting Data Augmentation strategies to the specific dataset and the objectives of the classification task.

References

[ACB17] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. PMLR, 2017.

[BPM04] Gustavo EAPA Batista, Ronaldo C Prati, and Maria Carolina Monard. A study of the behavior of several methods for balancing machine learning training data. *ACM SIGKDD explorations newsletter*, 6(1):20–29, 2004.

[CBHK02] Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357, 2002.

[EL21] Justin Engelmann and Stefan Lessmann. Conditional wasserstein gan-based oversampling of tabular data for imbalanced learning. *Expert Systems with Applications*, 174:114582, 2021.

[GAA+17] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C Courville. Improved training of wasserstein gans. *Advances in neural information processing systems*, 30, 2017.

[GPAM+20] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.

[HBGL08] Haibo He, Yang Bai, Edwardo A Garcia, and Shutao Li. Adasyn: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence)*, pages 1322–1328. Ieee, 2008.

[LHC22] Bohan Li, Yutai Hou, and Wanxiang Che. Data augmentation approaches in natural language processing: A survey. *Ai Open*, 3:71–90, 2022.

[LNA17] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017.

[LSBL18] Sayeri Lala, Maha Shady, Anastasiya Belyaeva, and Molei Liu. Evaluation of mode collapse in generative adversarial networks. *High performance extreme computing*, 2018.

[MG18] Agnieszka Mikołajczyk and Michał Grochowski. Data augmentation for improving deep learning in image classification problem. In *2018 international interdisciplinary PhD workshop (IIPhDW)*, pages 117–122. IEEE, 2018.

[MO14] Mehdi Mirza and Simon Osindero. Conditional generative adversarial nets. *arXiv preprint arXiv:1411.1784*, 2014.

[PCZ+19] Daniel S Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D Cubuk, and Quoc V Le. Specaugment: A simple data augmentation method for automatic speech recognition. *arXiv preprint arXiv:1904.08779*, 2019.

[Wil72] Dennis L Wilson. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, (3):408–421, 1972.

[XSNK20] Zhaozhao Xu, Derong Shen, Tiezheng Nie, and Yue Kou. A hybrid sampling algorithm combining m-smote and enn based on random forest for medical imbalanced data. *Journal of Biomedical Informatics*, 107:103465, 2020.

[XV18] Lei Xu and Kalyan Veeramachaneni. Synthesizing tabular data using generative adversarial networks. *arXiv preprint arXiv:1811.11264*, 2018.

Declaration of authorship

I hereby declare that I have prepared this thesis using no sources and aids other than those indicated. This thesis has not been submitted in its present or any similar form to any department or equivalent institution of any domestic or foreign university.

Selbstständigkeitserklärung

Hiermit erkläre ich, dass ich diese Arbeit unter Verwendung keiner anderen als der angegebenen Quellen und Hilfsmittel angefertigt habe. Die vorliegende Arbeit wurde weder in ihrer jetzigen noch in einer ähnlichen Form bei einem Fachbereich oder einer entsprechenden Institution einer in- oder ausländischen Hochschule eingereicht.

.....

Ort, Datum Unterschrift